THE AUSTRA LANGUAGE

Ian Marteens

Contents

WELCOME TO AUSTRA 1

THE DESIGN OF THE LIBRARY 1
VECTORISATION VERSUS TASKS 1
LINEAR ALGEBRA 2
MATRIX FACTORISATIONS 3
TIME SERIES 3
MEAN-VARIANCE OPTIMISER 3
POLYNOMIALS AND ROOT FINDING 3
FAST FOURIER TRANSFORM 4
LANGUAGE GOALS AND DESIGN 4
ARRAYS IN AUSTRA 5

LANGUAGE OVERVIEW 7

LEXICAL SYNTAX 7
NUMERIC LITERALS 7
STRING LITERALS 8
DATE LITERALS 8
COMMENTS 8
ROOT OBJECTS 9
GLOBAL VARIABLES 9
SESSION VARIABLES 10
CLASS METHODS AND CLASS CONSTANTS 11

PRIMITIVE TYPES 13

OPERATORS 13
COMPARISONS 14
COMPLEX PROPERTIES AND OPERATORS 15
INTEGER PROPERTIES 15
THE MATH CLASS 15
DATES 19
LOGICAL VALUES 20
CONDITIONAL EXPRESSIONS 20

DEFINITIONS 23

CREATING DEFINITIONS 23
DEFINITIONS CANNOT USE SESSION VARIABLES 24
DEFINITIONS MAY USE EXISTING DEFINITIONS 24
DETERMINISTIC CALLS 24

FUNCTION DEFINITIONS **25**
DESCRIBING A FUNCTION DEFINITION 26
TYPE NAMES IN AUSTRA 26

LOCAL VARIABLES **29**

LET CLAUSES **29**
SCRIPT-SCOPED LET CLAUSES **29**
LOCAL FUNCTION DEFINITIONS **30**

LAMBDA FUNCTIONS **31**

LAMBDA FUNCTIONS WITH ONE PARAMETER **31**
FUNCTION NAMES AS LAMBDAS 32
LAMBDA FUNCTIONS WITH TWO PARAMETERS **32**
BINARY OPERATORS AS LAMBDAS **33**
CAPTURED VARIABLES **33**
NESTED LAMBDAS **34**

TIME SERIES **37**

SERIES COME FROM EXTERNAL SOURCES **37**
ADDITIONAL INFORMATION IN SERIES 37
SERIES VERSUS VECTORS 37
CLASS METHODS 38
SERIES PROPERTIES 38
SERIES METHODS 40
OPERATORS 41
INDEXING AND SLICING **41**
LINEAR FITTING **41**
LINEAR MODELS **43**
STATISTICS ON TIME SERIES **44**
ACCUMULATORS 44
MOVING TIME WINDOWS 45
AUTOCORRELATION AND PARTIAL AUTOCORRELATION 46
AUTOREGRESSIVE MODELS 47
MOVING AVERAGE MODELS 49

VECTORS **53**

REAL VECTORS **53**
CLASS METHODS 53
VECTOR PROPERTIES 55
VECTOR METHODS 56
VECTOR OPERATORS 57

COMPLEX VECTORS **57**
COMPLEX VECTOR PROPERTIES 58
COMPLEX VECTOR METHODS 59
COMPLEX VECTOR OPERATORS 59
INTEGER VECTORS **59**
INTEGER VECTOR PROPERTIES 60
INTEGER VECTOR METHODS 61
DATE VECTORS **61**
DATE VECTOR PROPERTIES 62
DATE VECTOR METHODS 62
DATE VECTOR OPERATORS 62
INDEXING AND SLICING **63**
THE FAST FOURIER TRANSFORM **64**
FFT PROPERTIES AND INDEXERS 65

SEQUENCES **67**

DOUBLE-VALUED SEQUENCES AS LIGHT VECTORS **67**
SEQUENCE CONSTRUCTORS 67
THE UNFOLD SEQUENCE GENERATOR 68
METHODS AND PROPERTIES 69
INTEGER SEQUENCES **71**
CLASS METHODS 71
METHODS AND PROPERTIES 72
COMPLEX SEQUENCES **73**
CLASS METHODS 73
DELAYED EXECUTION **74**

MATRICES **77**

MATRIX CONSTRUCTION **77**
CLASS METHODS **77**
METHODS AND PROPERTIES **78**
MATRIX OPERATORS **79**
OPTIMISATIONS 80
INDEXING AND SLICING **81**
EIGENVALUES DECOMPOSITION **81**
LU FACTORISATION **83**
CHOLESKY DECOMPOSITION **84**

LIST COMPREHENSIONS **85**

SYNTAX **85**
TYPES IN LIST COMPREHENSIONS 85
GENERATORS 86
QUANTIFIERS IN LIST COMPREHENSIONS **86**

SPLINES **89**

CREATING SPLINES **89**
INDEXERS, METHODS, AND PROPERTIES **90**
INTERACTING WITH A SPLINE 91

MODELS **93**

MEAN VARIANCE OPTIMISER **93**
MORE CLASS METHOD OVERLOADS 95
ADDITIONAL CONSTRAINTS 96
LINEAR PROGRAMMING **97**

INDEX **99**

Welcome to AUSTRA

AUSTRA IS AN EFFICIENT mathematical library, written in C# and running on .NET Core, which is also used by a small functional language designed to handle financial series and common econometric models.

Both library and language, also support vectors, matrices, transforms and the most frequently used operations from linear algebra, statistics, and probability.

The library code is hardware-accelerated, using all resources provided by the CPU. The language compiler is also an optimising compiler, detecting common expression patterns and substituting them with more efficient method calls, whenever possible.

Austra contains three main components:

1. The Austra library, written in C# and .NET Core 8.
2. The Austra language: a simple formula-oriented language for testing and exploring the library.
3. The Austra application: a desktop application, written in WPF for Windows, providing a code editor with syntax highlighting and code completion, for trying the language.

The design of the library

The library has been designed as a set of mostly immutable types, to facilitate their concurrent use. Most of the methods are hardware-accelerated, either using managed references, SIMD operations, or both. Memory pinning, and raw pointers, have been reduced to a minimum, to ease the garbage collector's work.

Using immutable vectors, series and matrices has one drawback, and it is more stress for the garbage collector. For that reason, we offer combined operations, like other libraries do, to fuse several linear operations into one, when possible. The AUSTRA parser detects most of these cases for optimising them.

Vectorisation versus tasks

This might sound unintuitive, but it has been a guide when designing the Austra Library:

> Library code should make as much use as possible of hardware vectorisation, and only when this way is exhausted, you should turn to task concurrency if it makes sense.

My points:

- Library methods are usually short. For instance, the implementation of the `Map` method from a vector or sequence is embarrassingly parallel, but even a vector with 2048 items takes around a microsecond to be mapped. That is a very short time span to attempt parallelisation using tasks: the overhead of starting and waiting for finalisation trumps any gains of task parallelism.
- Neither vectorisation nor parallelisation play nice with modularity.
- We have chosen, for Austra, applying all possible vectorisations at the lowest level, and leaving task parallelism to higher level abstractions designed by the consumers of the class.

In any case, using task parallelisation with Austra is easy, in part due to classes implementing non-mutating operations.

Linear Algebra

Austra provides classes for dense vectors and matrices, for double-precision arithmetic. It also features an efficient complex vector type. Single-precision floats, complex and sparse matrices are planned for a future sprint. All operations takes advantage of C# operators when possible, so most of the operations are non-destructive.

There are three classes for representing matrices:

- `Matrix` is the general type that you will use the most.
- Lower triangular matrices are represented by the `LMatrix` type.
- Upper triangular matrices are represented by the `RMatrix` type.

The point with these two additional types is not to save space, since the underlying data structure is the same, but to provide a more efficient implementation of a couple of methods and operators. There are also logical advantages, regarding type safety since some decompositions returns triangular matrices.

As usual, matrix multiplication has been fully optimised using loop reordering and unrolling, blocking and hardware intrinsics, including fused multiply and add. There are variants for multiplying a matrix by another matrix transposed on-the-fly, for multiplying a vector by a transposed matrix and for accelerating linear combinations of vectors.

All these types are read-only structures, acting as a thin layer above C#'s arrays. Even the storage for a matrix is a one-dimensional array, since multidimensional

arrays in .NET are less optimised for bound checking, getting a managed reference and other low-level operations.

Matrix factorisations

Austra provides classes for the following matrix factorisations:

- Lower-Upper (LU) Factorisation.
- Cholesky Factorisation.
- Eigenvalues Decomposition (EVD).

Linear equation solving uses the LU factorisation internally.

Time series

The kernel of Austra was an implementation of the Mean-Variance optimiser. This means that time series were implemented before vectors and matrices.

Series are collections of pairs date/value, and they are sorted by date. Values can be used as vectors, but there are some differences. Vector operations check, at run time, that the operands have the same length. The same behaviour would be hard to enforce for series. On one hand, each series can have a different first available date. On the other hand, even series with the same frequency could have reported values at different days of the week or the month, and still, it could be interesting to mix them.

Mean-Variance Optimiser

A Mean-Variance Optimiser implementation is included, starting with the `MvoModel` class. This functionality is available at the formula language via the `model::mvo` class method.

The MVO model is rendered as an interactive model by the AUSTRA desktop application.

Polynomials and root finding

The `Polynomials` static class provides methods for polynomial evaluation and root finding. The `Solver` class implements a simple variant of the Newton-Raphson method for root finding.

There is also a `PolyEval` for evaluating polynomials using the Horner's method, and a `PolySolve` for analytically finding roots whenever possible, and using the eigenvalues of the Frobenius matrix in the general case. There is even a `PolyDerivative` for computing the derivative of a polynomial at a given abscissa.

Natural cubic splines have also been implemented, both for series and for functions, using a grid. You can even calculate the derivative of a spline at any point in the supported range.

Fast Fourier Transform

Austra implements a decent FFT algorithm, compared to most popular managed implementations. It uses the Cooley-Tukey algorithm, and it's optimised for small sizes. Small primes are handled either with Bluestein's or Rader's algorithm, depending on the size.

In any case, there is still room for improvement, and it's planned to be optimised in the future. AVX prefers structs of arrays over arrays of structures, and this preference obviously applies to complex arithmetic: it's more efficient to represent the real and the imaginary parts of a list of complex numbers in separate arrays.

Language goals and design

One of the motivations for creating Austra was having an easy-to-use language for testing and exploring functionality.

- The language should be mostly a functional one. Functional languages are expression-oriented, concise, and discourage mutability. These features match very well with the characteristics of the library.
- On the other hand, we did not want a complicated language with lazy evaluation and monads. I really like monads! Some of my best friends are monads! Jokes aside: I want Austra to be used by a wide base of professionals, instead of a selected group of freaks.
- A problem with R and MATLAB, which loosely fall in the same category as Austra, is the pollution of the global namespace. We wanted to avoid that. Instead of having a global `product` function that you could apply to a vector, we prefer a `product` method that is a feature of vectors.

There is also an important non-goal:

- We are not trying to substitute C# with Austra. AUSTRA, the language, is not supposed to be a Turing-complete programming language.

These are some consequences of the non-goal:

- We do not intend to write the Austra library in AUSTRA. That may be the goal for a next step, and, indeed, we already have some ideas and plans to do it. It would require, for make any sense, automatic vectorisation, for example.
- The type system of AUSTRA is very simple. There are no generic types. Type inference is primitive. Only a handful of classes from the library are fully exposed. And, in the current version, we still have no support for tuples.

To diminish the complexity of using the language, we also conceal some types as much as possible, to reduce the number of class names the programmer must remember. The language defines a small set of classes on which class methods, i.e., constructors and static methods, can be called. These classes are:

- `math`, for grouping global functions and variables.
- `matrix`, for dealing with all kinds of matrices.
- `vec`, `cvec`, `nvec` and `dvec`, for real, complex, integer, and date vectors.
- `seq`, `cseq` and `nseq`, for real sequences, complex sequences, and integer sequences.
- `series`, for time series.
- `spline`, for cubic splines.
- `model`, for mathematical models and tools.

Of course, AUSTRA manages a long list of types, from primitive types such as `date`, `bool` and `int`, to classes generated by transforms or matrix factorisations.

Arrays in AUSTRA

One example of how AUSTRA tries to hide complexity from the user is how arrays are handled by the language. Arrays pervade the library. You need an array of reals to create a vector, an array of vectors to create a matrix, and another array of series to create a covariance matrix. But arrays do not match well with a functional programming style.

What AUSTRA does is accept a variable number of parameters wherever a method needs an array parameter. This is, for instance, how AUSTRA creates a covariance matrix from a list of series:

```
matrix::cov(aaa, aab, aac);
matrix::cov(aaa, aab, aac, aad);
```

And this is how we efficiently create a linear combination of three series, including an "intercept," that is, a constant additional term in the linear combination:

```
series::new([0.1, 0.2, 0.3, 0.4], aaa, aab, aac);
```

In both cases, the implementing code receives an array of series as its last parameter, and AUSTRA automatically gathers all series at the end of the method call in a single array.

Language overview

AUSTRA IS A SMALL functional language designed to handle financial series and common econometric models. It also implements vectors, matrices, and the most frequently used operations from linear algebra, statistics, and probabilities.

AUSTRA formulas are efficiently parsed by a .NET Engine, and they are translated into fast-running native code that calls routines also implemented in .NET that take advantage of multicore systems and SIMD extensions.

This topic introduces the basic syntax of the language.

Lexical syntax

The lexical syntax of AUSTRA is remarkably like most programming languages:

- White space, including line returns, are completely ignored.
- Identifiers and keywords are key insensitive.
- Unicode characters are allowed in identifiers. So, yes: `τ = 2*π` is a valid expression. Of course, `pi` is also allowed, and the code editor helps while typing Greek characters.
- Semicolons (`;`) are mandatory for separating statements, but not as statement terminators.

Numeric literals

Integer and real numbers are represented as in most programming languages. Here are some examples:

```
2023;
1.0;
-0.1E-16
```

Number literals can be suffixed with a lower-case `i` to represent an imaginary value:

```
2.0i;
-3i
```

The identifier `i`, by its own, represents the imaginary unit:

```
1-3i = 1 - 3 * i
```

Complex numbers can also be created using the `complex` function:

```
complex(1, -3) = 1 - 3 * i;
complex(3) = 3 + 0i
```

Since `i` is not a keyword, you must be careful because it can be redefined as a user variable.

Complex can also be built using the polar notation:

```
polar(1, pi/2) -- Another way to write the imaginary unit.
```

String literals

String literals are enclosed by double quotes and cannot cross line boundaries.

```
"A simple string literal";
"A string literal with a quote: ""Wow!"". That was the quote."
```

Date literals

Date literals come in two flavours. A simple literal only includes the month and year, assuming the first day of the month:

```
jan20;
jul2021
```

Two-digit years are first interpreted as a year inside the XXI century. If the resulting date is more than 20 years ahead, 100 years are subtracted to that date. For instance:

```
jan20; -- January 1st, 2020
may42  -- May 1st, 1942
```

A day can be added to the constant core using this syntax:

```
6@jan20;
31@jul2021
```

Comments

Though we do not expect anyone to write hundreds of pages of AUSTRA script, we still support line comments for better documentation. Comments always start with two consecutive hyphens and extend to the next line feed or the end of the expression, whatever comes first:

```
-- A verbose version of math::min()
if aapl.mean < msft.mean then aapl.mean -- Another comment.
else msft.mean
```

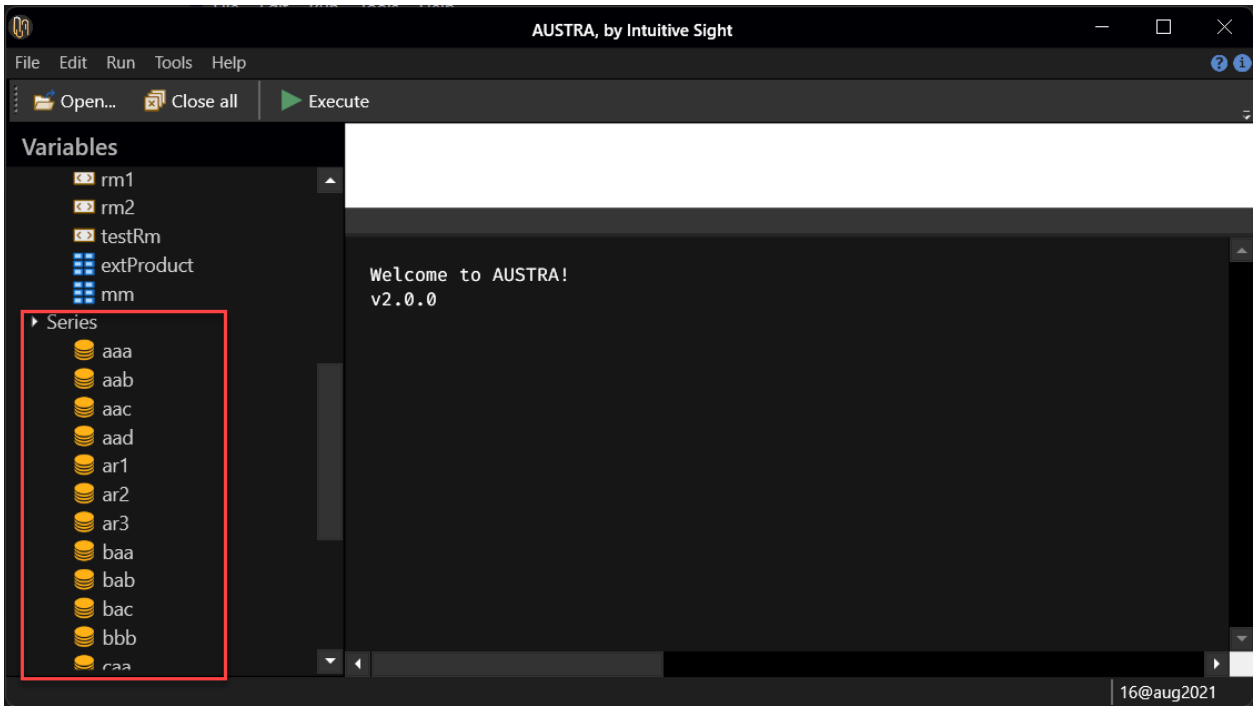
Root objects

Every AUSTRA expression must start with a root object. It could be either a global variable, a local variable, a class method, a class variable, or a code definition.

Global variables

Global variables come in two flavours: persistent variables and session variables. Persistent variables come mostly from an external source, like a JSON file, a database, or an external service. In this AUSTRA version, those persistent variables are always time series because they have a predictable serialisation format. This design decision, of course, may change at some point in the evolution of the library.

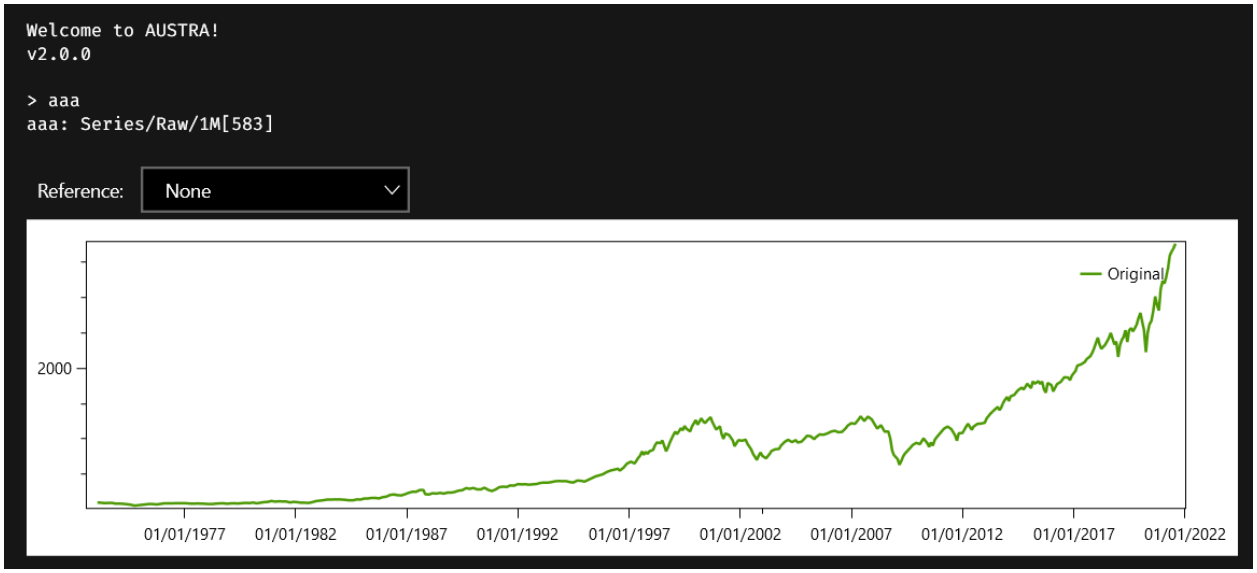
For instance, when I open the AUSTRA application in my system, it automatically loads a set of series and definitions that are stored in a subfolder *Austra* of my *Documents* folder, in a file named *data.austra*, and its main windows looks something like this:



Persistent series are shown below a *Series* node. I can type the name of any of these variables in the **`Code Editor`**:

```
aaa
```

When I press `F5`, AUSTRA translates the expression and immediately shows the content of the `aaa` series:



Session variables

Session variables, as the name indicates, are defined inside a user session, and die with the session. They are defined and removed using the `set` statement:

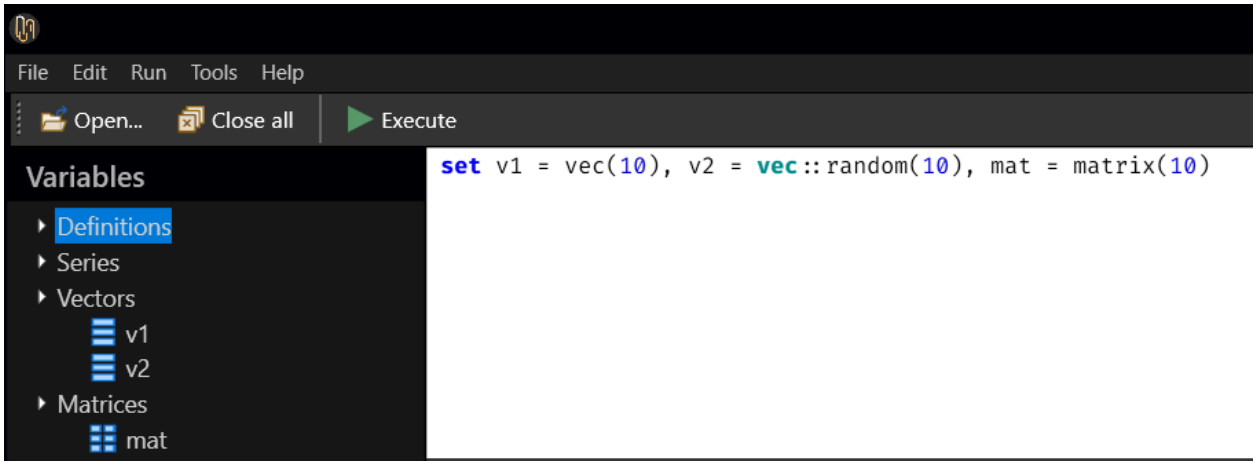
```
set v1 = [1, 2, 3, 4, 5];
set v2 = v1.map(x => 1 / x);
v2.plot;
-- v1 is removed now:
set v1;
-- v2, however, persist for the rest of the session.
v2.plot;
-- You can assign more that one session variable in a statement.
set p2 = 2pi, p3 = 3pi;
```

Only the value of the variable is stored, but not the formula that was used to calculate that value. This means, for example, that every use of the session variable will return the same value, even if the value was created using a random number generator:

```
-- v1 is created using random numbers:
set v1 = vec::random(10);
-- Every use of v1 always returns the same vector:
v1 = v1;
-- This is in contrast with the behavior of local variables.
let v2 = vec::random(10);
-- This expression will return false:
v2 = v2
```

Local variables are explained in the next section.

Session variables appear in the **`Variables`** panel, each one inside a node according to their types:



Class methods and class constants

Class methods in AUSTRA correspond both to constructors and static methods in traditional OOP languages, like C#.

Let's start with some variables:

```
i = math::i;
e = math::e;
pi = math::pi and pi = math::π
```

The same equivalence is valid for what we normally would consider "global functions":

```
exp(π*i);
math::exp(math::pi * math::i)
```

Those global functions and constants are considered by the compiler as belonging to the `math` for avoiding problems if any of these symbols is redefined as a persistent or session variable.

Of course, there are more classes than `math`, and we can use their class methods for creating new objects:

```
matrix::random(10);
vec::new(10);
vec(10)
```

As the last example shows, when you call a `new` method on a class, you can omit the `::new` part and use just the class name as synonym.

Primitive types

AUSTRA ARITHMETIC IS basically the same as on most programming languages. The language supports:

- 32 bits integers, represented by the `int` type.
- 64 bits integers, represented by the `long` type.
- 64 bits double-precision reals, represented by the `double` type.
- 2x64 bits double-precision complex values, represented by the `complex` type.

Smaller arithmetic types are automatically converted by the compiler to bigger types when required: `int` to `double`, `double` to `complex`, and even `int` to `complex`. Double values can be converted into integer values using the `toInt` property, as in `pi.toInt`.

Operators

These are the operators available for integers and reals:

`+`	Addition. Can also be used as a unary operator.
`-`	Subtraction. Can also be used as a unary operator for negation.
`*`	Multiplication.
`/`	Both real and integer division.
`%`	Both integer and real remainders.
`^`	Power: `2^3 = 8`, `9^0.5 = 3`.

Most of them may also be used with complex numbers.

Though the power operator works both for integer, real and complex numbers, the compiler optimises the cases when the power is 2, 3 and 4, so equalities like this exactly holds:

```
i^2 = -1
```

The multiplication operator can be elided when the first operand is a real or an integer and it is immediately followed by an identifier:

```
2pi = 2 * pi
2x^2 + 3x + 1 = 2*x^2 + 3*x + 1
1/2x = 1 / (2*x)
```

AUSTRA also recognises a superscript *2* (²) as an operator to square a value:

```
2x² + 3x + 1 = 2*x^2 + 3*x + 1
```

The AUSTRA code editor simplifies typing this operator with the key's combination (CTRL+G, 2).

Comparisons

These operators are used for comparing all compatible operands:

`=`	Equality.
`!=`	Inequality.
`<>`	A synonym for the inequality operator.
`<`	Less than.
`<=`	Less than or equal to.
`>`	Greater than.
`>=`	Greater than or equal to.
`<-`	Belongs to. Right side must be a vector or a sequence.
`∈`	A fancy synonym for the membership operator (`<-`).

The membership operator can be used with sequences, vectors, matrices, and series:

```
34 <- [1..100];
0 <- vec::random(1024)
```

When the right side of the membership operator is a time series, the left operand may be either a real or a date:

```
0.0 <- appl.rets;
1@jan2020 <- appl
```

Comparisons can be fused for numeric operands using the following syntax:

```
sqrt(pi) <= pi <= pi²
```

Fused ranges only require combining same-direction comparisons. For instance, `<=` and `<` are compatible, but `<` and `>` are not.

Complex properties and operators

When you have a complex value in your hands, you can drill into it using a dot and a property name, to extract information about the poor little value:

`real`	The real part of the complex.
`imaginary`	The imaginary part of the complex.
`magnitude`	A magnitude, i.e., the distance to `complex(0, 0)`.
`phase`	The phase, in radians.

```
let c = complex(3, 4) in
    c = c.real + c.imaginary * i
```

If typing `magnitude` is too hard for your nerves, you can use `mag` as an accepted synonym. `real` can be shortened to `re`, and `imag` and even `im` can be used instead of `imaginary`. Since my heart is cold and empty for `phase`, on the other hand, there is no diminutive for that fellow.

In addition to the usual operators, there is a suffix operator for conjugating a complex value:

`'`	Unary suffix operator for complex conjugation.

The `'` operator is also used for conjugating complex vectors and transposing a matrix.

Integer properties

Integer values support the `even` and `odd` properties for easy testing of parity:

```
iff(e.toInt.even, "Truncated to 2", "Rounded to 3")
```

The math class

The `math` class groups methods and properties dealing with arithmetic operations. Most of these features come straight from the C#'s `Math` and `Complex`, but it also incorporates other functions that are used in statistics and probabilities.

Our `math` is special in that the class prefix is assumed when not present in a function or property call:

```
-- Write like this, if you are a sucker for pain.
math::sin(math::pi/4) = math::sqrt(2)/2;
-- Standard people use this style.
sin(pi/4) = sqrt(2)/2
```

Why, for the love of Mike, have we sunken all those definitions inside the `math` class? It is easy to explain with two points: we did not want to pollute the global name space with lots and lots of symbols in the first place. Some mathematically oriented languages do just this: everything is a global function, so, at some point, you must come up with very clever but cryptic names for your own stuff. Nonetheless, we can omit the class name for the most used names. The second point is related: somebody can shadow one of these global names, such as `i` or `max`. In those cases, you still have the long and winding road of prefixing the shadowed name with its class name, and nothing is lost.

These are the methods or functions provided by this class. Most of them work with both integer, real, and complex parameters:

`abs`	Absolute value
`acos`	The angle whose cosine is the specified parameter.
`asin`	The angle whose sine is the specified parameter.
`atan(x)` `atan(x, y)`	The angle whose tangent is the specified parameter. The version with two parameters is equivalent to Atan2.
`beta(x, y)`	Bi-parametric Euler integral of the first kind.
`cbrt`	Cubic root.
`complex`	Creates a complex number from one or two real values.
`cos`	The cosine function.
`cosh`	The hyperbolic cosine function.
`erf`	The error function.
`exp`	The exponential function.
`gamma`	The gamma function: an extension of factorials for real numbers.
`lnGamma`	The natural logarithm of the gamma function.
`log`	The natural logarithm function.
`log10`	Base 10 logarithms.

`max`	The maximum of its two parameters. It also works with dates.
`min`	The minimum of its two parameters. It also works with dates.
`ncdf`	Normal cumulative distribution function.
`polar`	Creates a complex from its circular coordinates.
`probit`	The inverse of the cumulative of the standard normal distribution.
`round(d)`	Rounds a real to the nearest integer.
`round(d, i)`	Rounds a real to the given number of decimals.
`sign`	Returns the sign of the argument.
`sin`	The sine function.
`sinh`	The hyperbolic sine function.
`sqrt`	The square root.
`tan`	The tangent function.
`tanh`	The hyperbolic tangent function.
`trunc`	Truncates a real value.

These are the properties (parameter-less functions) and constants provided by `math`:

`e`	Euler's constant.
`i`	The imaginary unit.
`maxInt`	The maximum value that is representable in an integer.
`maxReal`	The maximum value that is representable in a real.
`minInt`	The minimum value that is representable in an integer.
`minReal`	The minimum value that is representable in a real.
`nrandom`	A random number from the standard normal distribution.
`pearl`	An Easter Egg. Just try me!

`pi, π`	Don't be irrational: be transcendent.
`random`	A random number from a uniform distribution between 0 and 1.
`tau, τ`	Twice π.
`today`	The current date.

Polynomials and solvers

These methods are also defined inside the `math` class, so they can be used without explicitly writing the class prefix:

`solve`	A simple Newton-Raphson solver. See below for details.
`polyEval`	Evaluates a polynomial given a real or complex argument.
`polyDerivative`	Evaluates the first derivative of a polynomial at a real or complex argument.
`polySolve`	Calculates all the roots of a polynomial.

The Newton-Raphson solver is a function accepting from three up to five arguments:

```
solve(x => sin(x) - 1, x => cos(x), 0, 1e-9, 100)
```

The first two arguments are lambda functions: one for the function we want to solve for a root, and the second for the first derivative of that function. Please note that `solve` does not verify that the lambda function and its derivative lambda match. The third argument is required and represents the initial guess to start running the algorithm. Again, a bad guess may make the algorithm fail.

The fourth and fifth arguments can be omitted. The fourth parameter is the desired accuracy, and when omitted, it defaults to `1e-9`. The last parameter is the maximum numbers of iterations, which by default is `100`.

The `polyEval` function takes either a complex or a real as its first argument, and a list of coefficients, either in a single vector or as a list of real values, and evaluates the polynomial at the supplied value:

```
let x1 = complex(-1, sqrt(2)), x2 = x1';
-- 1, 2, 3 represents the polynomial x² + 2x + 3
polyEval(x1, 1, 2, 3);
-- Coefficients can be grouped in a vector.
polyEval(x2, [1, 2, 3]);
```

The inverse of `polyEval` is the `polySolve` function. It takes either a vector or a list of reals and considers them as the coefficients of a polynomial. The first value is the coefficient of the highest degree term. For instance, the vector `[1, 2, 3, 4]` stands for the polynomial $x^3 + 2x^2 + 3x + 4$. This function can throw an exception if it does not know how to manage a given polynomial, or when there are no available roots. The returned value is always a complex vector, even when all roots are reals. For instance:

```
polySolve(1, 2, 3)
```

You can check the accuracy of the answers from the solver using this trick:

```
let poly = [1, 2, 3] in
  polySolve(poly).all(c => abs(polyEval(c, poly)) <= 1e-15)
```

Of course, the accuracy of the roots may vary according to the polynomial.

A close relative of `polyEval` is `polyDerivative`, which calculates the derivative of a given polynomial at the specified argument:

```
let v = [1, 2, 3, 4];
polyEval(2, v) = 26;
polyDerivative(2, v) = 23
```

`polyDerivative` can be useful when finding a real root for a polynomial using the Newton-Raphson algorithm.

Dates

Dates in AUSTRA are represented by the `date` type and stores the number of days since Jan 1st, 1900. Dates support these properties:

`day`	Gets the day of month, starting by 1.
`dow`	Gets the day of the week.
`isLeap`	Is the year from the date a leap one?
`month`	Gets the month of the date, starting with 1.
`toInt`	Converts the date to a signed integer.
`year`	Gets the year of the date.

These two methods allow adding either a positive or a negative number of months or years to a date:

`addMonths`	Adds a positive or negative number of months to a date.
`addYears`	Adds a positive or negative number of years to a date.

Adding or subtracting days from a date is achieved with these operators:

`+`	Adds days to a date. The left operand must be a date.
`-`	Subtracts days from a date. The left operand must be a date. It can also be used to find the difference in days between two dates.

Logical values

Logical values are represented by the `bool` data type. Variables and parameters of this type hold one of these two constants: either `false` or `true`.

Operators acting on logical values resemble more the good-old Pascal operators than the C/C++/C# ones. It's a matter of personal preference, of course, but also of readability:

`not`	Logical negation.
`and`	Logical conjunction.
`or`	Logical disjunction.

The precedence of these operators is the standard one. Negation binds first, then conjunction, and finally disjunction.

Conditional expressions

Since AUSTRA is a functional language, it doesn't have "statements". However, it provides an `if/then/else` ternary operator, equivalent to the also included `iff()` function:

```
if aapl.mean < msft.mean then aapl.mean else msft.mean
```

Of course, the above expression is just a pedantic way to write `min(aapl.mean, msft.mean)`. It can also be written using `iff()` this way:

```
iff(aapl.mean < msft.mean, aapl.mean, msft.mean)
```

Most of the times, the more verbose ternary operator is easier to read. The ternary operator has another advantage: you can chain more than one conditions and responses using the `elif` keyword.

```
let x = random;
if x < 0.1 then "Too low!"
elif x < 0.5 then "A little low"
elif x < 0.9 then "A little high"
else "Too high!"
```

Definitions

CODE DEFINITIONS ARE formulas saved for future use. They are saved and loaded from any persistent storage used by AUSTRA. You can define either definitions without parameters, which act like macros, or parametric definitions, which are the equivalent of user-defined functions.

Creating definitions

Definitions are created using the `def` statement:

```
def cxMvo = model::mvo(sm_ret, sm_cov, sm_low, sm_high)
```

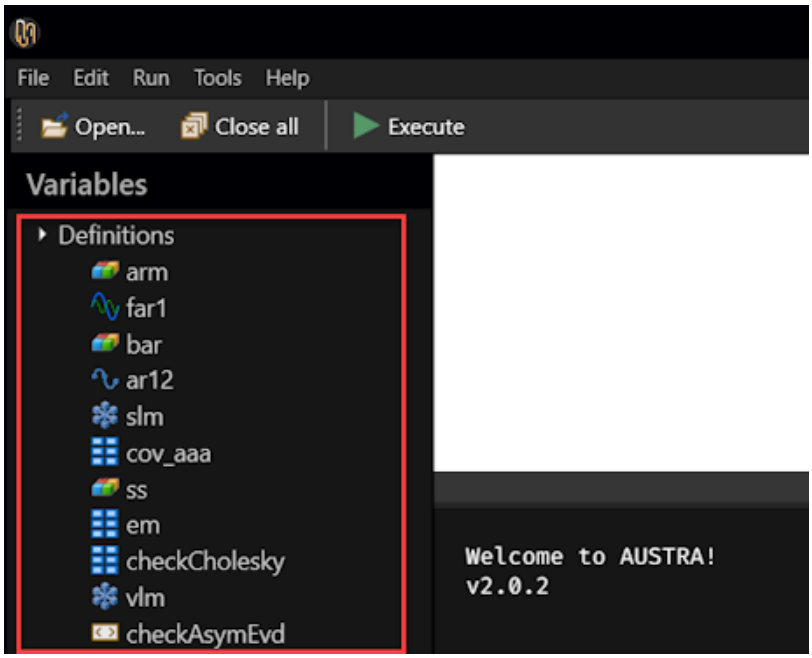
A description can be associated to a definition using the following syntax:

```
def cxMvo:"MVO Model" = model::mvo(sm_ret, sm_cov, sm_low, sm_high)
```

Removing an existing definition is achieved with the `undef` command:

```
undef cxMvo
```

In the AUSTRA desktop application, definitions appear in the **`Variables`** panel, inside a **`Definitions`** node:



Definitions cannot use session variables

Code definitions must respect a rule: they cannot reference session variables. This sequence of commands is invalid:

```
set vector = [1, 2, 3, 4];
def fact4 = vector.product;        -- Invalid code definition.
def fact5 = [1, 2, 3, 4].product; -- This, however, it's fine.
```

The reason behind this constraint is that session variables only store their current values, but not the formula that generated that value.

Definitions may use existing definitions

A code definition may refer to an existing definition. For instance:

```
def sm_cov = matrix::covariance(aapl, msft, esx, dax);
def sm_ret = [1, 0.9, 1.2, 0.8];
def cxMvo = model::mvo(sm_ret, sm_cov, vec(4), vec::ones(4))
```

In this case, removing either `sm_cov` or `sm_ret`, would also remove `cxMvo`.

Deterministic calls

Let's say we write this definition:

```
def extProduct = vec::random(4) ^ vec::random(4)
```

This definition calls twice a class method that creates a random vector. The caret operator, `^`, combines those two vectors in a 4x4 matrix. Executing these definitions two times in a row gives, as expected, different results:

```
> extProduct
ans ∈ ℝ(4×4)
0.416065  0.493621  0.412334  0.0249965
0.390261  0.463007  0.386762  0.0234462
0.377909  0.448353   0.37452  0.0227041
 0.49103   0.58256  0.486626  0.0295002

> extProduct
ans ∈ ℝ(4×4)
0.0251534  0.0182728  0.0452763  0.00933612
0.0374942  0.0272379    0.06749   0.0139167
0.0555746  0.0403725   0.100035   0.0206275
0.0256057  0.0186015  0.0460906  0.00950403
```

That is the expected behaviour. However, this could be inconvenient to test properties of the result. For instance, we could want to check the determinant of the product, or that a double transpose works fine:

```
extProduct = expProduct''; -- Double transpose.
(extProduct * extProduct).det - extProduct.det ^ 2
```

AUSTRA assumes that, inside a formula, all parameter-less definition calls must return the same value. For that purpose, the two above formulas are internally rewritten as:

```
let x = extProduct in x = x'';
let x = extProduct in (x * x).det - x.det ^ 2
```

The parser creates a local variable under the hood for evaluating the definition just once inside the current formula.

This automatic caching only takes place for parameter-less definitions. If you want to disable this behaviour, just add an exclamation sign right after the definition identifier, when using the definition:

```
-- This first expression returns true.
extProduct = extProduct;
-- This second expression returns false.
extProduct = extProduct!;
-- This expression also returns false.
extProduct! = extProduct
```

Function definitions

A definition can also have parameters, for defining a function. For instance, the factorial of an integer can be defined this way:

```
def fact(n: int) = iff(n <= 1, 1, [2..n].prod)
```

The above definition is non recursive. Recursive functions must declare their return type:

```
def recFact(n: int): int =
    if n <= 1 then 1 else n * recFact(n - 1)
```

You can use local variables when defining a function:

```
def mcd(a, b: int): int =
    let m = a % b in iff(m = 0, b, mcd(b, m))
```

And you can also define auxiliary functions inside a function definition:

```
def fact(n: int) =
    let f(n, acc: int): int = iff(n <= 1, acc, f(n - 1, n * acc)) in
        f(n, 1)
```

In this case, the inner function `f` is the one that is directly recursive. The outer function does not need to declare its return type.

Describing a function definition

A permanent description can be attached to a function definition using the same syntax as before:

```
def fact: "Iterative factorial"(n: int) =
    iff(n <= 1, 1, [2..n].prod)
```

The description will be serialized and saved in whichever data storage Austra uses.

Type names in AUSTRA

These are the types that can be explicitly used for parameters and return types in function definitions:

`bool`	The logical data type.
`int`	32-bit integers.
`long`	64-bit integers.
`real`	Double precision reals.
`date`	Austra dates.
`string`	Strings.
`complex`	Double precision complex values.
`series`	Time series.
`matrix`	Dense double precision matrices.
`vec, cvec, ivec`	Real, complex, and integer vectors.
`dvec`	Date vectors.
`seq, cseq, iseq`	Real, complex, and integer sequences.

Arrays can be specified adding two brackets after a type name. Function types follows this convention:

```
-- A function that receives a real, and returns a real:
real => real
-- Receives an integer and a vector, and returns a real:
int => vec => real
```

For instance, this definition allows to apply a function twice to an argument:

```
def twice(x: real, f: real => real) =
    f(f(x))
```

This function can be called like this:

```
twice(1, sin)
```

Local variables

AUSTRA IS A FUNCTIONAL language, so it has a functional technique for declaring what in a procedural language would be temporal or local variables.

LET clauses

The functional technique for declaring local variables in a formula is the `let` clause.

```
let m = matrix::lrandom(5),
    m1 = m * m',
    c = m1.chol in
    (c * c' - m1).aMax
```

In the above example, a lower triangular random matrix is computed, and it is multiplied by its transpose. Then, the Cholesky transform is calculated and finally we check that the transform is valid, evaluating the absolute maximum of the matrix difference.

The `m`, `m1` and `c` variables only exist while the formula is being evaluated. As the example shows, each variable defined in the let clause can use any of the previously declared variables in the same clause.

Script-scoped LET clauses

When writing more than one statement in a script, `let/in` clauses are valid only for the statement they precede, but not for other statements:

```
let m = matrix::lrandom(5),
    m1 = m * m',
    c = m1.chol in
    (c * c' - m1).aMax;
-- The next statement cannot use "m".
m
```

If you need a local variable to be available for all statements that follow in a script, you must use a variant of `let` which does not terminate with an `in` keyword, but with a semicolon:

```
let m = matrix::lrandom(5);
-- Now, "m" is available for the rest of the script.
let m1 = m * m',
    c = m1.chol in
    (c * c' - m1).aMax;
```

```
-- The next statement is valid.
m
```

Note

Some functional languages, as Haskell, feature another construct for abstracting sub-expressions. Haskell, for instance, offers both **let** and **where**. **let** is located before the expressions that make use of it, and **where** comes after the main expression.

In AUSTRA, we prefer **let**, for the sake of Code Completion. So far, I cannot think of any use for **where** that cannot be solved better with **let**.

Local function definitions

Functions can be defined in `let` clauses. For instance:

```
let mcd(a, b: int): int = if a % b = 0 then b else mcd(b, a % b) in
    mcd(80, 140)
```

In the above example, the function is defined in a `let`/`in` clause, but it could also be defined as a script-scoped local function.

Note

Since `mcd` is recursive, its return type must be declared in the function header.

Function definitions may have their own local variables, as in this variant of the above example:

```
let mcd(a, b: int): int =
    let m = a % b in iff(m = 0, b, mcd(b, m)) in
        mcd(80, 140)
```

This way, we save one evaluation of the remainder.

Local functions may also be declared inside other functions. For instance, this code defines a function for the factorial, but uses an intermediate function that can be evaluated using tail recursion, for efficiency:

```
let fact(n: int) =
    let f(n, acc: int): int = iff(n <= 1, acc, f(n - 1, n * acc)) in
        f(n, 1);
fact(10)
```

Please note that the `in` keyword applies to the right-side of the definition of `factorial`. The `let` clause that defines `factorial`, on the contrary, is a script-level clause, with no associated `in`.

Lambda functions

LAMBDA FUNCTIONS ARE inline-defined anonymous functions that can be used as parameters in normal methods and class method calls.

Lambda functions with one parameter

Series and vectors, for instance, have an `all` method to check if all their numeric values satisfy an arbitrary condition. The condition is the only parameter of the method and must be passed as a lambda function. Let us say we have an `aapl_prices` persistent variable holding a series of prices. We can verify that all those prices are positive using this formula:

```
aapl_prices.all(x => x >= 0)   -- It should return true.
```

The above formula checks whether all values in the price series are non-negative. That's the role of the `all` method, which checks that all values in a series satisfies a given predicate. The way we state the predicate to be satisfied is using this syntax:

```
x => x >= 0
```

This can be read as "given an arbitrary value x, check that it is non-negative". We can use all for any other purpose, such as checking that all values in a series lie inside the (0, 1) interval:

```
prices.all(value => 0 < value < 1)
```

Notice that in this new example, we have used another name for the "arbitrary given value": `value` instead of `x`. This renaming has no effect in the formula.

This example shows how to use the related method `any`:

```
prices.any(x => x >= 1)
```

In this case, we are checking whether exists at least one value in prices that is above 1.

Both `any` and `all` require a predicate as argument: a formula that given an arbitrary value, returns `true` or `false`. The `map` method, instead, requires a more general function that converts a real value into another one. Let us say we want to limit values from a series, so that no one is greater than 1000:

```
prices.map(x => min(x, 1000))
```

In all cases, the type of the parameter of the lambda is determined by the method the lambda is passed, and so is the returned type. AUSTRA adds any required conversion, as when a double is required for the result and an integer expression is being returned. Regarding the name of the lambda's parameter, you can use any name you like, keeping in mind that it will shadow any predefined identifier inside the lambda function's body.

Function names as lambdas

In many cases, you need a lambda that takes a single parameter to transform it into another value from the same type. For instance, the sine function can be approximated using a spline over a uniform grid like this:

```
let s = spline(0, 2*pi, 1024, x => sin(x)) in
    s[pi/4]
```

The above code can be shortened to this:

```
let s = spline(0, 2*pi, 1024, sin) in
    s[pi/4];
```

Or even this if you need to qualify the function name for any reason:

```
let s = spline(0, 2*pi, 1024, math::sin) in
    s[pi/4];
```

Since `sin` is a function with a single parameter and no parameters are supplied, the compiler understands that the function must be used to create a mono-parametric lambda, returning a real value.

Lambda functions with two parameters

Some methods require lambda arguments with more than one parameter. When a lambda requires two or more parameters, their names must be enclosed inside parenthesis, and must be separated by commas.

```
iseq(1, 10).reduce(0, (x, y) => x + y)
```

`zip` can act on arguments with different lengths, so it only acts in the common part of both. It generates a new series, vector or sequence, and each item will be the combined value created by the lambda function. In the above example, it will be the maximum price for each common date.

Binary operators as lambdas

You can also use a binary operator as a shortcut for a lambda definition. This code uses the `reduce` method on a sequence of integers for summing all items in the sequence

```
aapl_prices.zip((x, y) => max(x, y))
```

You can substitute the lambda definition with a reference to the binary operator, including its class name:

```
iseq(1, 10).reduce(0, int::+)
```

This trick, so far, only works with binary operators.

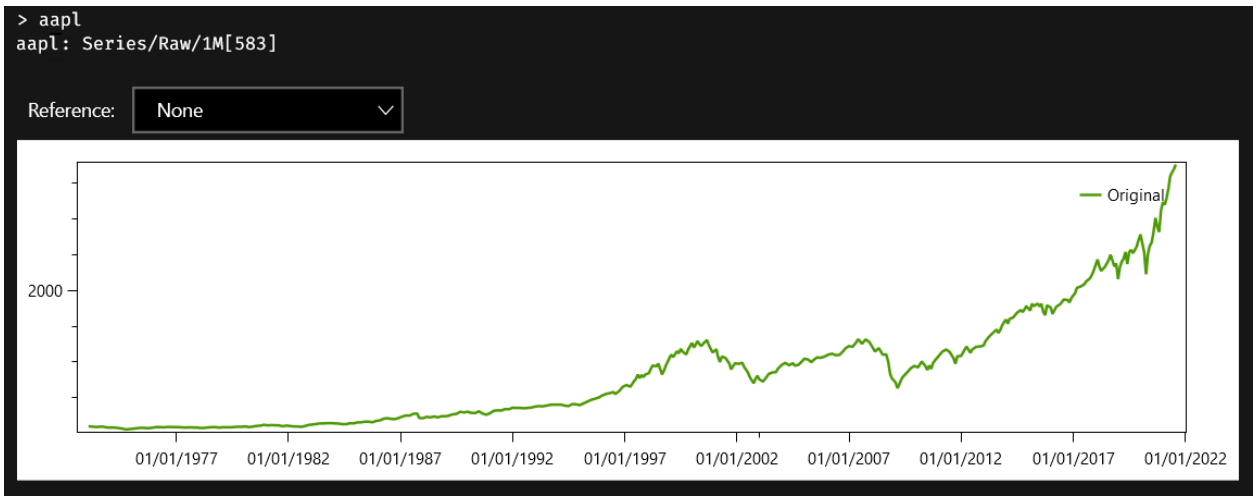
Captured variables

The `ncdf()` method of a series takes a real value and classifies it according to its position in the normal distribution implicitly defined by the series. It is, by definition, a value between 0 and 1. Even better, `ncdf()` is monotonic: $x < y \Rightarrow s.ncdf(x) < s.ncdf(y)$. All this means that this method is a convenient way to compress an arbitrary series, so all their values lie between 0 and 1, while preserving the shape of the series.

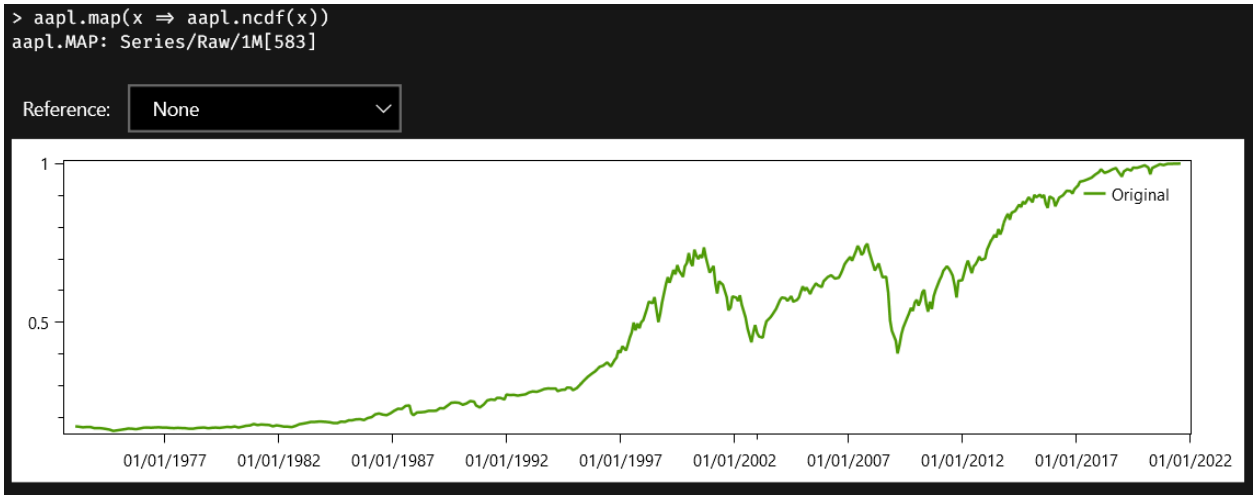
This formula does the trick:

```
aapl.map(x => aapl.ncdf(x))
```

Nothing remarkable here: `aapl` is a global identifier, and it should not surprise us that we can use it both in the main formula and in the nested lambda. This is the original series:



And this is the compressed series:



Please note that the main difference between both charts is the range of values.

What if what we really wanted was the compressed series with the simple returns of prices? Not a big deal. This, obviously, works:

```
aapl.rets.map(x => aapl.rets.ncdf(x))
```

But we can do it much better, using a `let` clause:

```
let a = aapl.rets in
  a.map(x => a.ncdf(x))
```

Though `a` is a local variable defined in the main body of the formula, we still can reference it from our nested lambda function. This way, we avoid recalculating the returns of the series in the lambda's body.

Note

The `series.ncdf(x)` method assumes that values in the series can be described by a normal distribution. This is seldom true.

A most useful related method is `series.movingNcdf(points)`, which calculates the `ncdf` for each value in the series but calculates the two parameters that defines a normal distribution from a configurable interval of points preceding each calculation.

Nested lambdas

Another kind of capture takes place when a lambda function is defined inside another lambda. This formula finds all prime numbers up to 100, and uses nested lambdas:

```
iseq(2, 100).filter(x => iseq(2, x - 1).all(div => x % div != 0))
```

Note

The above code also uses sequences for generating a range or list of integers.

The underlined text is a definition of a lambda that is being used as the argument of the `filter` method. It's a function with a single parameter `x`. Note, however, that inside that lambda, we call another method that has its own lambda function, using the parameter `div`. The inner lambda can use both its own parameter `div`, but it also can use `x`, defined by the outer function.

Time series

THE MOST IMPORTANT data type in AUSTRA is the *time series*: a sorted collection of pairs date/value.

Series come from external sources

Since time series represent data from the real world, most of the time, series come from persistent variables that can be stored in an external file or database and may be periodically updated, either by AUSTRA or by another process.

Additional information in series

Since one of the goals of AUSTRA is to deal with financial time series, there is a few optional properties that can be stored in a series:

Name

The name of the series is the name that is used by the parser to locate a series. For this reason, the series' name must be a valid identifier.

Ticker

However, it's frequent for series to be identified by traders by their tickers, which is the name assigned by the provider of the series. A ticker is not necessarily a valid identifier, so we provide two different fields, one for the name and the second for a ticker. A ticker can be empty.

Frequency

Each series has an associated frequency, which can be daily, weekly, biweekly, monthly, bimonthly, quarterly, semestral, yearly, or undefined. The library, at run time, checks that both operands in a binary operation have always the same frequency.

Series type

In addition, each series has a type that can be either `Raw`, `Rets`, `Logs`, `MixedRets`, or `Mixed`.

Series versus vectors

Vector operations check, at run time, that the operands have the same length. The same behaviour would be hard to enforce for series. On one hand, each series can have a different first-available date. On the other hand, even series with the same

frequency could have reported values on different days of the week or the month, and still, it could be interesting to mix them.

So, the rules for mixing two series in an operation are:

- They must have the same frequency, and their frequencies are checked at runtime.
- However, they may have different lengths. If this is the case, the shorter length is chosen for the result.
- The points of the series are aligned according to their most recent points.
- The list of dates assigned to the result series is chosen arbitrarily from the first operand.

Class methods

There is only one constructor for series:

`series::new`	Creates a linear combination of series.

The first parameter of `series::new` must be a vector of weights, and from that point on, a list of series must be included. This class method creates a linear combination of series. The length of the weights vector can be equal to the number of series or the number of series plus one. For instance:

```
series([0.1, 0.9], aapl, msft);
-- The above code is equivalent to this:
0.1 * aapl + 0.9 * msft
```

If we add another item to the vector, it will act as an independent term:

```
series([0.5, 0.1, 0.9], aapl, msft);
-- The above code is equivalent to this:
0.5 + 0.1 * aapl + 0.9 * msft
```

Series properties

These properties are applied to instances of series:

`acf`	The Autocorrelation Function (ACF).
`amax`	Gets the maximum of the absolute values.
`amin`	Gets the minimum of the absolute values.
`count`	Gets the number of values in the series.
`dates`	Dates of the series as a vector of dates.

`fft`	Gets the Fast Fourier Transform of the values.
`first`	Gets the first point in the series (the oldest one).
`fit`	Gets a vector with two coefficients for a linear fit.
`kurt`	Get the kurtosis.
`kurtp`	Get the kurtosis of the population.
`last`	Gets the last point in the series (the newest one).
`linearFit`	Gets a line fitting the original series.
`logs`	Gets the logarithmic returns.
`max`	Get the maximum value from the series.
`mean`	Gets the average of the values.
`min`	Get the minimum value from the series.
`movingRet`	Gets the moving monthly/yearly return.
`ncdf`	Gets the percentile of the last value.
`pacf`	The Partial Autocorrelation Function (ACF).
`perc`	Gets the percentiles of the series.
`random`	Creates a random series from a normal distribution.
`rets`	Gets the linear returns.
`skew`	Gets the skewness.
`skewp`	Gets the skewness of the population.
`stats`	Gets all statistics in one call.
`std`	Gets the standard deviation.
`stdp`	Gets the standard deviation of the population.
`sum`	Gets the sum of all values.
`type`	Gets the type of the series.
`var`	Gets the variance.

`varp`	Gets the variance of the population.
`values`	Gets the underlying vector of values.

Series methods

These are the methods supported by time series:

`all`	Checks if all items satisfy a lambda predicate.
`any`	Checks if exists an item satisfying a lambda predicate.
`ar`	Calculates the autoregression coefficients for a degree.
`arModel`	Creates a full AR(p) model.
`autocorr`	Gets the autocorrelation given a lag.
`corr`	Gets the correlation with a series given as a parameter.
`correlogram`	Gets all autocorrelations up to a given lag.
`cov`	Gets the covariance with another given series.
`ewma`	Calculates an Exponentially Weighted Moving Average.
`filter`	Filters points by values or dates.
`indexOf`	Returns the index where a value is stored.
`linear`	Gets the regression coefficients given a list of series.
`linearModel`	Creates a full linear model given a list of series.
`ma`	Calculates the moving average coefficients for a degree.
`maModel`	Creates a full MA(q) model.
`map`	Pointwise transformation of the series with a lambda.
`movingAvg`	Calculates a Simple Moving Average.
`movingNcdf`	Calculates a Moving Normal Percentile.
`movingStd`	Calculates a Moving Standard Deviation.
`ncdf`	Gets the normal percentile for a given value.
`stats`	Gets monthly statistics for a given date.

`zip`	Combines two series using a lambda function.

Operators

These operators can be used with time series:

`+`	Adds two series, or a series and a scalar.
`-`	Subtracts two series, or a series and a scalar. Also works as the unary negation.
`*`	Multiplies a series and a scalar for scaling values.
`/`	Divides a series by a scalar.
`.*`	Pointwise series multiplication.
`./`	Pointwise series division.

Indexing and slicing

Points in a series can be accessed using an index expression between brackets:

```
aapl[0];
aapl[appl.count - 1].value = aapl.last.value;
aapl[^2] = aapl[aapl.count - 2]
```

Series also supports extracting a slice using dates or indexes. In the first case, you must provide two dates inside brackets, separated by a range operator (`..`), and one of the bounds can be omitted:

```
aapl[jan20..jan21];
aapl[jan20..15@jan21];
aapl[jan20..];
aapl[..jan21]
```

The upper bound is excluded from the result, as usual. Date arguments in a series index do not support the caret (`^`) operator for relative indexes. When using numerical indexes in a slice, the behaviour is like the one of vectors:

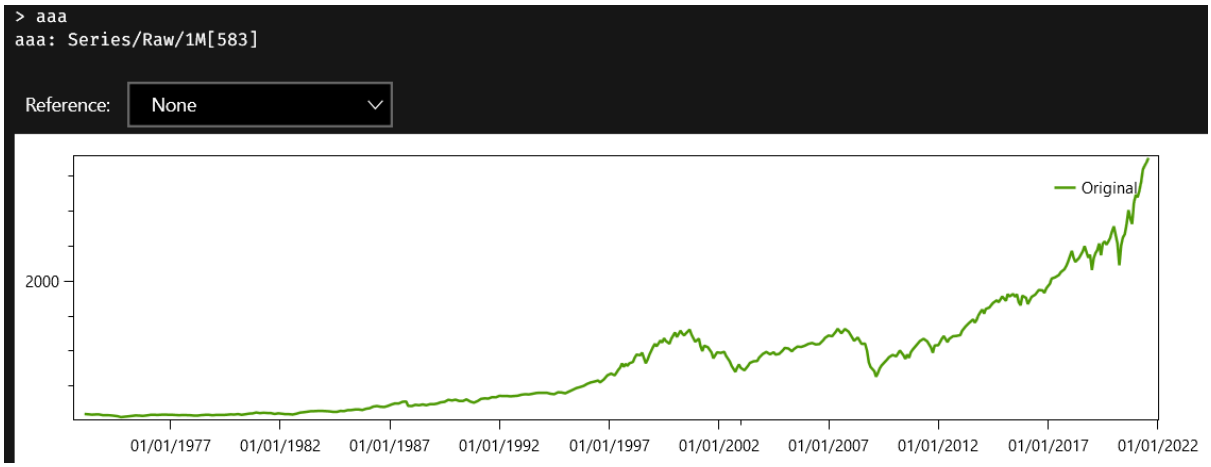
```
aapl[1..aapl.count - 1].count = aapl[1..^1].count
```

Linear fitting

When you face a time series for the first time, the first thing you want is to decompose the series into all its identifiable components. The series may be the sum of a linear or quadratic process, it may have seasonal variations or any kind of periodic

variation, it may show signs of a stochastic process such as an autoregressive or moving average process and, of course, there will be almost always random noise in the raw data.

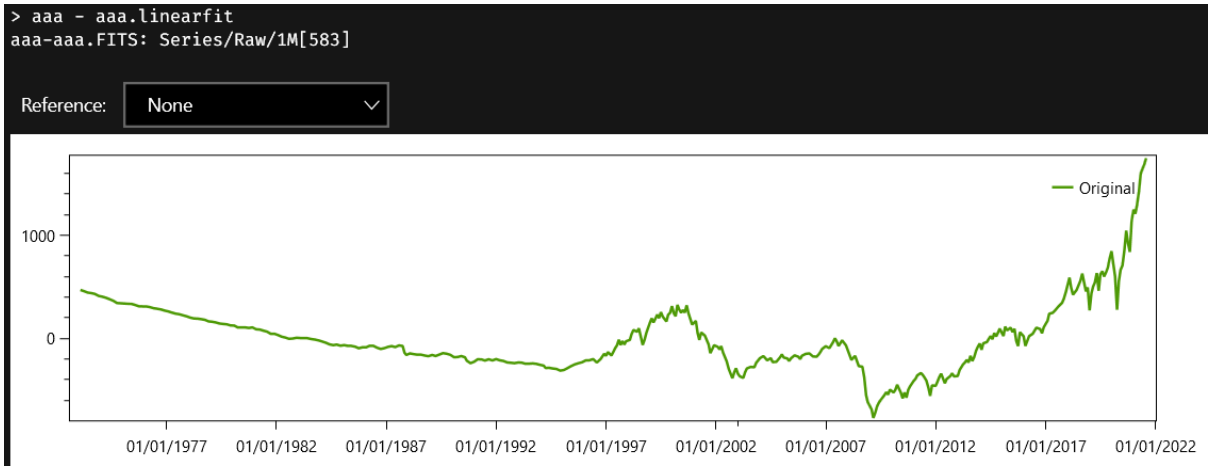
The most easily identifiable component is perhaps a linear trend in the data. Look at the chart for this raw series with monthly sampling:



If you apply the `fit` method to this series, the answer will be two numbers in a real vector:

```
aaa.fit
-- This is the answer:
ans ∈ ℝ(2)
0.134716  -97404.4
```

The first number in the vector is the slope, and the second number is the intercept, that is, the value when the argument of the corresponding line is zero. If you wanted to look at the inferred line, you could execute the `linearFit` property on `aaa`, which creates a series with the same date, but with values from the fitted line. You can also subtract `aaa.linearFit` from `aaa`, to see the part of the data that cannot be explained by a simple line model:



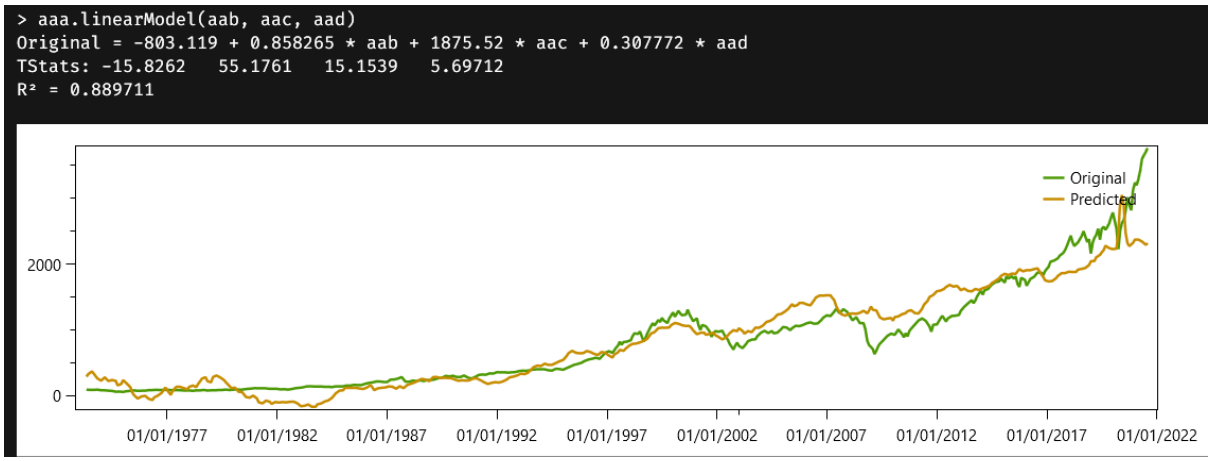
Austra uses Ordinary Least Squares (OLS) to find the coefficients of the fitting line.

Linear models

If your data cannot be easily explained using a simple line, you could try another approach: explaining a series as a linear combination of other existing series. Let us say that we want to explain the `aaa` series using three other series. This is the formula we need:

```
aaa.linearModel(aab, aac, aad)
```

An instance of the `LinearModel` class is created, and this is how the Austra Desktop application shows it:



The most important data is contained in the first line of the answer:

$$aaa = -803.12 + 0.858 * aab + 1875.52 * aac + 0.308 * aad$$

That is how the series to be explained can be approximated with the other three predicting series. Coefficients are calculated to minimise the OLS of the difference between the prediction and the original.

The second line give us the t-statistics for the relevance of each coefficient. Note, for example, that the most relevant coefficient is the one corresponding to the `aab` series, even though the coefficient for `aac` is greater. The reason is that values from `aac` are smaller than values from `aab`. The next line gives us the R^2 statistics, also known as *goodness of fit*, which is the quotient between the explained variance and total variance. The closer R^2 is to one, the better the explanation is.

Finally, the application shows a chart including the original and the predicted series.

In case you only need the coefficients of the model, you can call the `linear` method on `aaa`, using the same parameters as before. `linear` just returns a vector with the coefficients used in the model.

Statistics on time series

A fair share of the properties and methods implemented by series have to do with statistics, either of the whole series or of partial samples from the series. Most of these properties and methods are shared with real vectors and sequences, for obvious reasons.

The next few sections deal with time series features that compute statistics for a time series.

Accumulators

The `stats` property returns an object from the C#'s `Accumulator` class that holds statistics on all samples from the series.

The `Accumulator` class defined by the Austra library, implements a running accumulator that calculates and updates the most important statistics estimators as we keep adding values from a dataset, using the well-known Welford algorithm. Our implementation of Welford's algorithm takes advantage of SIMD instructions from the CPU, when available. Since it is a fast implementation, the result returned by the `stat` property of a series is always computed when the time series is created. Persistent series are created when the Austra Desktop application starts up, or when a series is retrieved for the first time from an external service or database. This way, you can always call `stats` without concerns about efficiency, and the same is valid on any property derived from the running accumulator.

Most of the properties of stats are also available as direct properties of the series, for convenience. They are:

`count`	Gets the number of values in the series.
`kurt`	Get the kurtosis.
`kurtp`	Get the kurtosis of the population.
`max`	Get the maximum value from the series.
`mean`	Gets the average of the values.
`min`	Get the minimum value from the series.
`skew`	Gets the skewness.
`skewp`	Gets the skewness of the population.
`var`	Gets the variance.
`varp`	Gets the variance of the population.

Remember that most of all these properties are just estimators, and that their accuracy depends on the number of samples. Skewness and kurtosis, for example, needs more than a thousand samples for a ballpark estimate, at least according to my own experience.

```
> aaa.stats
Count: 583
Min  : 54.8
Max  : 3764.1
Mean : 822.8759862778732
Var  : 589666.4405563715
StdDv: 767.8974153859169
Skew : 1.243177799325843
Kurt : 1.306668284237245
```

One nice property about running accumulators is that you can combine two of them easily and efficiently using the plus operator:

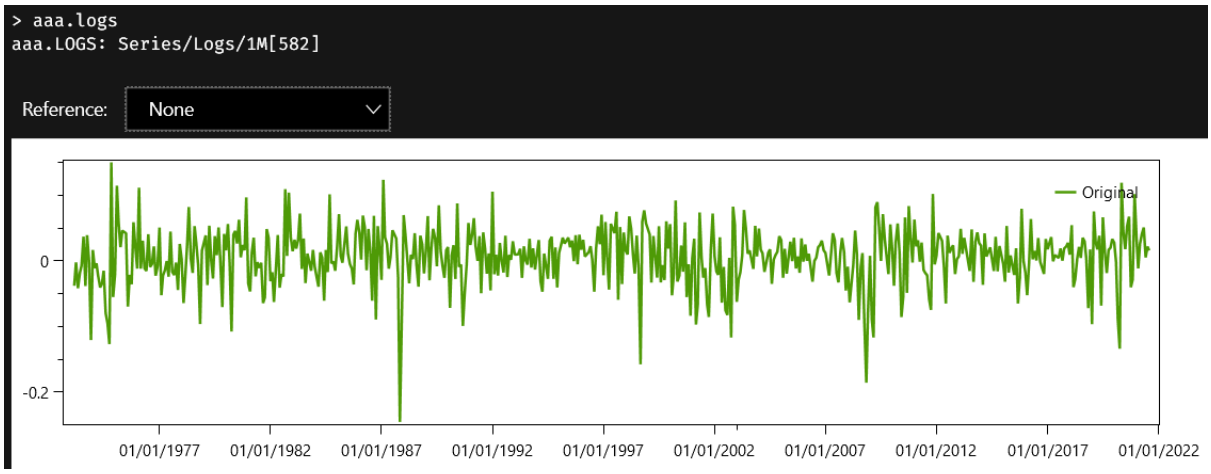
```
aaa.stats + aab.stats
```

A `stats` property is also implemented by real and integer vectors and sequences.

Moving time windows

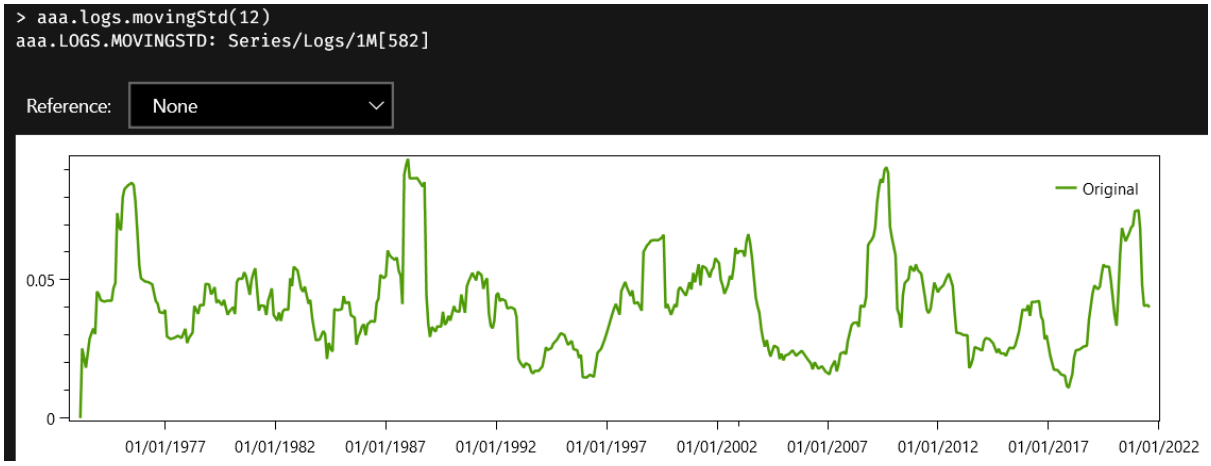
A series like our friend `aaa` is classified as a *raw* series. It starts with low values, and, despite random oscillations, it has a definite upward trend. If we take the mean of the first half of the series, it will be wildly different from the other half's mean. The average value is not a significant property of the series.

Part of the problem has to do with the fact that `aaa` probably represents a random walk. The most probable underlying process that generates a series like this works by throwing a die at each step and deciding how much we must increase or decrease the current value. We can focus, however, on how these variations behave, by transforming the series into a series of *returns*. Series have two properties for this task: `rets`, for linear or ordinary returns, and `logs`, for logarithmic returns. This chart shows the logarithmic returns of `aaa`:



Now we have a series with a uniform mean throughout its lifetime, but another problem has surfaced. The time range we sample has a significant impact on the variation in the returns. In other words, different segments have different standard deviations or variances.

That is the reason why there are series methods like `movingAvg`, `movingStd`, and `movingNcdf` that calculate those statistics over a sliding window of samples. Each of these methods requires the number of samples to define the moving window. Since `aaa` is a monthly series, the following chart displays the accumulated standard deviation for a moving period extending from one year.



The related property, `movingRet`, can be used on raw and return series and averages the return according to the sampling frequency of the series. In this case, we do not supply the size of the sliding window, but it is inferred from its type and frequency.

Autocorrelation and partial autocorrelation

There are two important diagnostic functions on series: the autocorrelation function, also known by the acronym ACF, and the partial autocorrelation function, also known as PACF.

The ACF is defined as the Pearson correlation between a signal and a delayed copy of the signal. The argument of the function is the delay between the samples, and since we are dealing with discrete signals, the type of the argument is an integer value.

There are formulas that work better with stationary series, that is, series with uniform statistical properties, widely speaking. Most financial series are not stationary when the sampling time is long enough, as we will see soon. Our algorithm does not assume stationarity and is based on the Wiener-Khinchin theorem, which relates the autocorrelation function with the power spectral density via the Fourier transform. Internally, Austra calculates a Fast Fourier Transform on a padded version of the series, using the next available power of two size for speed.

The Partial Autocorrelation Function, or PACF, is closely related but must not be confused with the ACF. The PACF measures the direct correlation between two lags, without accounting for the transitive effect of any intermediate lags. Our PACF implementation calculates the ACF as a prerequisite and then performs the Levinson-Durbin algorithm on the ACF to remove those spurious effects from the lags in-between.

Autoregressive models

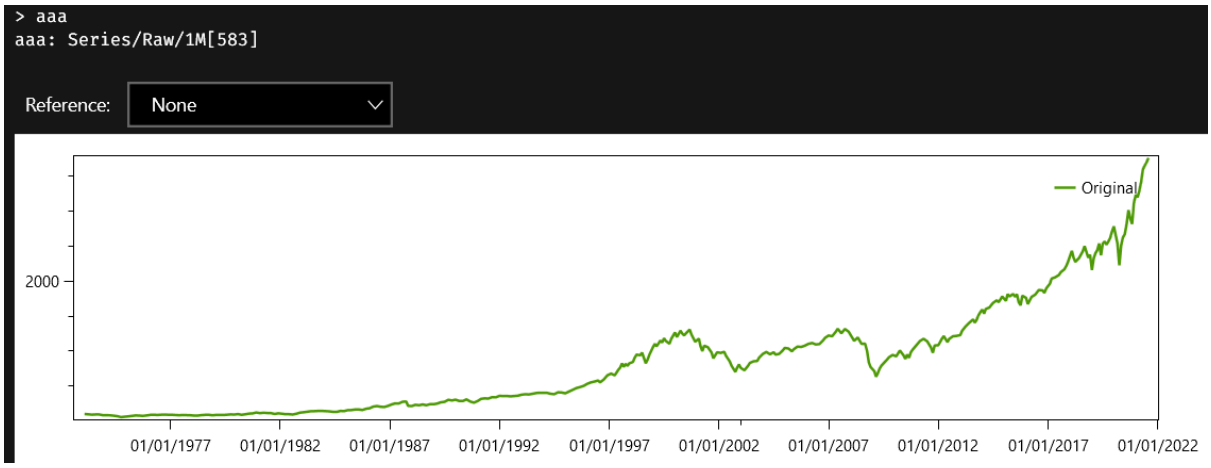
The autoregressive model is one of the simplest stochastic models that can generate a time series. If we denote as x_t the value of a series at a time or step t, an autoregressive model of order p is a process generated by this formula:

$$x_t = \sum_{i=1}^{p} \varphi_i x_{t-i} + \varepsilon_t$$

The ε_t term is a random value taken from any distribution, not necessarily a standard one. It is only required that the mean of ε_t be zero. If the order of the model, p, is zero, what remains is just white noise. And things start getting interesting when $p > 0$, because each term start been influenced by a subset of the preceding terms in the series.

The easiest way to generate an autoregressive model for testing is using class methods from real sequences. What series provide are methods for estimating parameters for an autoregressive model, assuming that the series has been generated by such a model. The `arModel` method estimates coefficients and includes some useful statistics with the output. The `ar` method is a leaner version of `arModel` that only returns coefficients. Finally, you can use the `pacf` property to check if an autoregressive process would be a good guess about how the series has been generated.

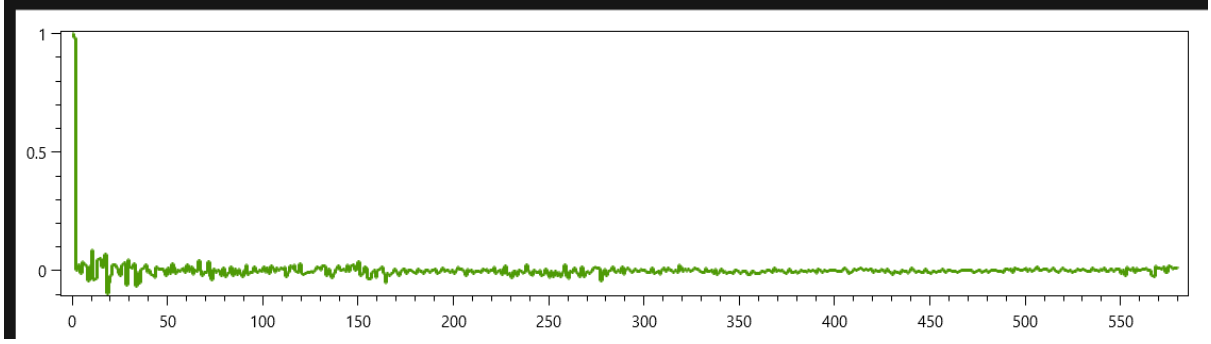
Let us take as example our good-old `aaa` series:



This is not a stationary series, and it looks more like a random walk. But an autoregressive process can yield random walks instead of stationary series when the sum of the coefficients is high enough. We will start by calculating the Partial

Autocorrelation Function of the samples we have. Theory says that partial autocorrelations for an autoregressive model fall to zero after a few lags. And that is just what we see when we plot the full PACF of `aaa`:

```
> aaa.pacf
Series/Raw[581]: PACF(aaa)
```



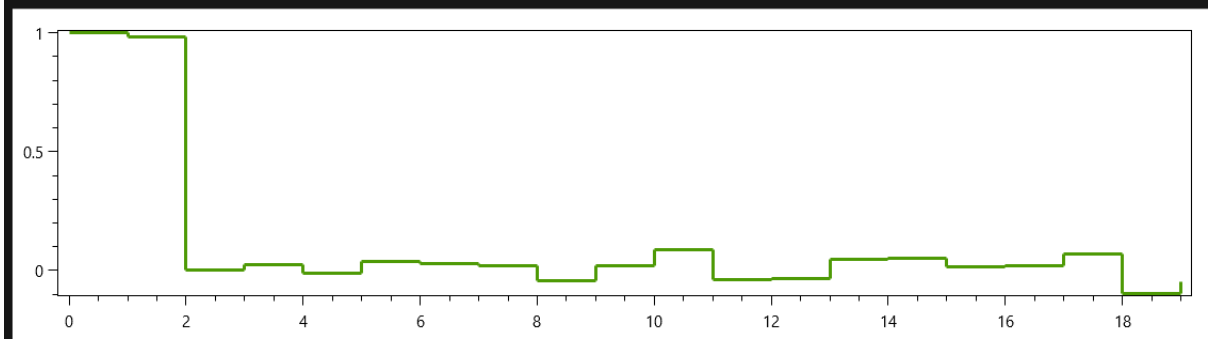
The return type of properties like `acf` and `pacf` is `series<int>`: instead of the usual pairs containing date/value, here we return pairs containing lag/value. Unfortunately, the control used for the chart does not have all the whistles and bells we would want. So, lets manually zoom on the first lags, to see what is happening.

Since series stores their values in reverse order, what we really want is a slice from the end of a series, so we will evaluate this formula:

```
aaa.pacf[^20..^0]
```

And this is the new chart we get:

```
> aaa.pacf[^20..^0]
Series/Raw[20]: PACF(aaa).SLICE
```



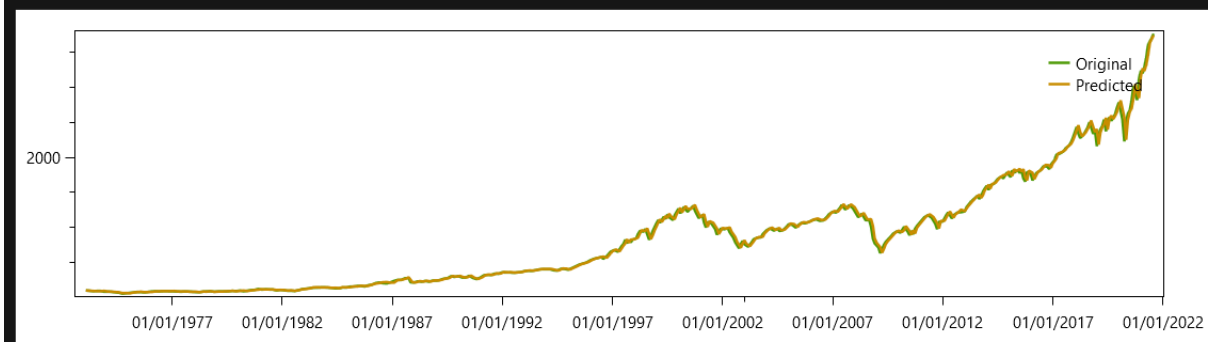
The first value of both the ACF and the PACF functions corresponds always to the lag zero, so it is always one. The value for the lag one is near one, and then, all the rest of the partial autocorrelations are negligible. We will bet that we can model the series with an autoregressive model of degree 1, and we are pretty sure that the coefficient will be high enough to generate a random walk instead of a stationary series. We will use the more nuanced method `arModel` to get as information as possible:

```
aaa.arModel(1)
```

And voilà, here we have the estimated model:

```
> aaa.arModel(1)
Coefficients: ans ∈ ℝ(1)
1.01017

R²: 0.9963
```



As we already suspected, the coefficient is greater than unity. The `r2` property of the model is the same *goodness of fit* we have already seen with linear models. It is the quotient of the explained variance over the total variance, and it is high enough for the model to be considered a good one.

The chart plots both the original series and the "predicted" one. As a word of caution, don't be fooled by the word "prediction": what we are forecasting is just one step forward, assuming the historically attested levels. It would be impossible, as it stands to reason, to generate the whole series from an initial level and the autoregressive law.

Note

Austra estimates coefficients for AR models using the so-called Yule-Walker equations.

Moving Average models

Another common process that generates a time series is the algorithm known as Moving Average. A Moving Average process of order q, often referred as MA(q), is defined by the following formula:

$$x_t = \mu + \sum_{i=1}^{q} \theta_i \, \varepsilon_{t-i} + \varepsilon_t$$

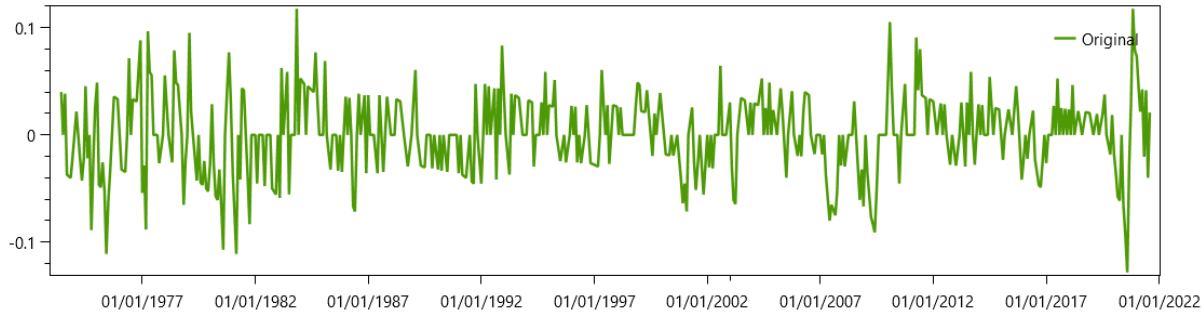
This is a very different beast than the autoregressive models we have already seen. All the ε_i terms still refer to random variables centred around zero. We also have a new term, μ, which is interpreted as the mean of the series. What makes an MA model different from an AR model is that what is propagated to successive steps is not the actual value at a past time, but the error term introduced in a previous step.

Pure Moving Average series are stationary series. Since I do not have a good real candidate at hand, I will use a transformed series as the source of the MA example. I have an `aac` series, and I will take its linear returns as my original samples. Even

then, the resulting series is not a stationary one, so the match will not be perfect. This is how I get the linear returns from a time series:

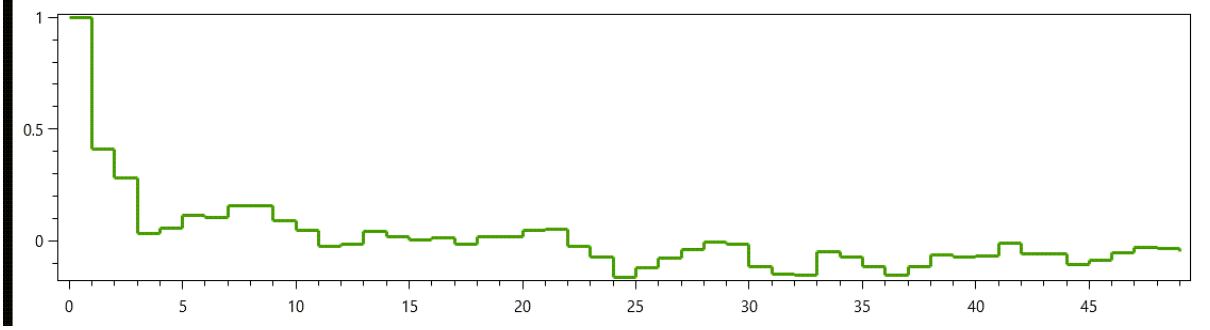
```
aac.rets
```

And this is the corresponding chart:



For MA series, we must use the ACF instead of the PACF. These are the fifty first lags of the ACF:

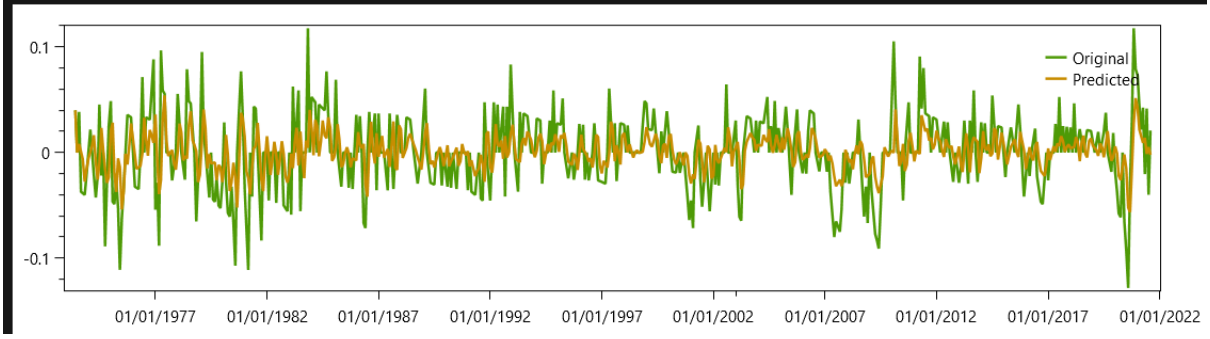
```
> aac.rets.acf[^50..]
Series/Rets[50]: CORR(aac.RETS).SLICE
```



This time, the cutoff is not as clear as before. Let us start by trying an MA(2) model:

```
> aac.rets.maModel(2)
Coefficients: ans ∈ ℝ(2)
0.389073  0.349279

μ: 0.000508758
R²: 0.216392
```



It could have been worse. The `r2` value is nothing to write home about. Note that, this time, we also have the μ parameter for the mean of the series. We could keep

raising the number of degrees of the model for a better match, but we would soon meet diminished returns. Remember that the original series was not a stationary one.

Note

MA models are way harder to estimate than the simpler AR model. They depend on past "errors," which are not directly observable but must be inferred from the samples.

Vectors

AUSTRA PROVIDES DOUBLE-PRECISION vectors, identified by the class `vec`, complex double-precision vectors, `cvec`, vectors of integers, `ivec`, and date vectors, `dvec`. All these data types are implemented using dense storage.

Real vectors

A vector is constructed by listing its components inside brackets:

```
[1, 2, 3, 4]
```

Commas are mandatory for separating items, and the compiler always ignores white space and line feeds.

Bracket lists can be also used to concatenate the content of several vectors, and you can add scalars to the mix:

```
let v1=[1, 2], v2=[3, 4];
-- Returns a vector with 4 items.
[v1, v2];
-- This also is accepted:
[[1, 2], v2];
-- Scalars can also be added.
[0, v1, pi, v2, tau];
```

Class methods

Vectors can also be created using these class methods:

`vec::new`	Overloaded constructor.
`vec::ones`	Creates a vector filled with ones.
`vec::random`	Creates a vector with random values from a uniform distribution.
`vec::nrandom`	Creates a vector with random values from a normal standard distribution.

These are the overloads supported by `vec::new`:

```
-- Creates a vector with 10 items, all of them zeros.
vec::new(10);
-- Remember that ::new can be omitted!
vec(10);
```

```
-- Creates a vector like [1 2 3 4 5 6 7 8 9 10]
vec(10, i => i + 1)
```

The last example shows how to create a vector using a lambda function parameter. This is a more sophisticated example of using a lambda to initialise items in a vector:

```
-- Mimics a periodic function.
vec(1024, i => sin(i*pi/512) + 0.8*cos(i*pi/256))
```

`vec::new` can also be used to create a linear combination of vectors:

```
vec([0.5, 0.1, 0.7, 0.2], v1, v2, v3)
```

The first parameter contains weights, and the remaining parameters are the vectors that will be linearly combined. If there is an extra value in the weights, as in the example, it is used as an independent term. The above expression is equivalent to this one:

```
0.5 + 0.1 * v1 + 0.7 * v2 + 0.2 * v3
```

Please note that the parser can detect some code patterns and optimise expressions automatically. For instance, for vectors, the parser recognises these patterns:

```
vector1 * scalar + vector2;
scalar * vector1 + vector2;
scalar1 * vector1 + scalar2 * vector2;
```

All these expressions are reduced to calls to one of the overloads of either `MultiplyAdd` or `Combine2`. These methods are internally optimised to use a single temporary buffer, instead of the two buffers of a naïve implementation, and both use FMA fused operations when available. Of course, the method underlying the above presented `vec::new` constructor is even better optimised and runs several times faster than even the better versions of lineal composition.

Another experimental optimisation substitutes non-destructive operators by operations that directly modify the internal buffer of one of the operands:

```
vec::random(10) + vector2;
```

It does not matter where `vector2` comes from. The first operand is a “new” vector, created on the fly for this formula, and its buffer will not survive beyond this formula. The compiler reckons it is safe to substitute the non-destructive addition with a version that leaves the result in the buffer of the first operand.

Even negations can be optimised with an in-place operation when acting on a non-shared object:

```
-(vec::random(10) + vector2);
```

Since the result of the inner sum is already recognised as a non-shared object, the usually non-destructive negation is substituted by its destructive version.

Note

The in-place operations optimisation is done by the compiler instead of the runtime because this way it is performed on the safe side. We cannot destroy the buffer of a shared instance, so we check first if the candidate to the in-place operation is a non-shared instance, as determined by the compiler. Excluded expressions include function parameters, **let** and session variables.

Vector properties

Properties and methods or vectors are like the ones from series. As a rule, almost all code in series that do not need to take dates into account, is implemented via the corresponding vector code, which is heavily optimised, and hardware accelerated. These are the properties supported by a vector instance:

`abs`	Gets a new vector with absolute values.
`acf`	The Autocorrelation Function (ACF).
`amax`	Gets the maximum of the absolute values.
`amin`	Gets the minimum of the absolute values.
`distinct`	Gets a new vector with the unique values from the original.
`fft`	Gets the Fast Fourier Transform of the values.
`first`	Gets the first item in the vector.
`last`	Gets the last item in the vector.
`length`	Gets the number of values in the vector.
`max`	Get the maximum value from the vector.
`mean`	Gets the average of the values in the vector.
`min`	Get the minimum value from the vector.
`norm`	Gets the Pythagorean norm of the vector.
`pacf`	The Partial Autocorrelation Function (ACF).
`plot`	Shows the vector in a chart.

`prod`	Multiplies all items in the vector.
`reverse`	Creates a new vector with items in reverse order.
`sort`	Gets a new vector with its items sorted.
`sortDesc`	Gets a new vector with items sorted in descending order.
`sqr`	Gets the scalar product of the vector with itself.
`sqrt`	Gets a new vector with the square root of each item.
`stats`	Gets all statistics in one call.
`sum`	Gets the sum of all values.

Vector methods

These are the methods supported by a vector instance:

`all`	Checks if all items satisfy a lambda predicate.
`any`	Checks if exists an item satisfying a lambda predicate.
`ar`	Gets the autoregression coefficients for a given `p`.
`arModel`	Creates a full AR(p) model.
`autocorr`	Gets the autocorrelation given a lag.
`correlogram`	Gets all autocorrelations up to a given lag.
`filter`	Filters items by value.
`find`	Like `filter` but returns a sequence with the indexes.
`indexOf`	Returns the first index where a value is stored.
`linear`	Gets the regression coefficients given a list of vectors.
`linearModel`	Creates a full linear model from a list of vectors.
`ma`	Estimates coefficients for an MA(q) model.
`maModel`	Creates a full MA(q) model.
`map`	Point-wise transformation of the items in a vector.
`reduce`	Reduces all items in a vector to a single value.

`zip`	Combines two vectors using a lambda function.

Vector operators

Real-valued vectors support the basic repertoire of operators:

`+`	Adds two vectors, or a vector and a scalar.
`-`	Subtracts two vectors, or a vector and a scalar. Also works as the unary negation.
`*`	Multiplying two vectors represents the inner vector product, returning a number. A vector multiplied by a scalar is a vector scaling operation.
`/`	Divides a vector by a scalar.
`.*`	Point-wise vector multiplication.
`./`	Point-wise vector division.
`^`	Outer product for two vectors, returning a matrix.

The outer product of vectors x_i and y_j returns the matrix with components $m_{i,j} = x_i y_j$. This expression call:

```
[1,2,3]^[4,5,6]
```

computes this matrix:

```
ans ∈ ℝ(3×3)
 4   5   6
 8  10  12
12  15  18
```

Complex vectors

There's no special syntax for complex vector literals, but complex vectors can be easily created using the `cvec::new` class method and one or two vector constructors:

```
cvec::new([1, 2, 3, 4], [4, 3, 2, 1]);
-- ::new can be omitted.
cvec([1, 2, 3, 4], [4, 3, 2, 1])
```

These class methods are available for creating complex vectors:

`cvec::new`	Overloaded constructor (see below).

`cvec::random`	Creates a complex vector with random values from a uniform distribution.
`cvec::nrandom`	Creates a complex vector with random values from a normal standard distribution.

These are the overloads supported by `cvec::new`:

```
-- Creates a complex vector with 10 zeros.
cvec(10);
-- Creates a complex vector from one real vector.
cvec([1, 2, 3]);
-- Creates a complex vector from two real vectors.
cvec([1, 2, 3], [3, 2, 1]);
-- Creates a complex vector with a lambda function.
cvec(10, i => polar(2π*i/10));
-- The lambda function includes access to the complex vector.
cvec(100, (i, v) => polar(2π*i/10) - 0.01 * i * v{i-1})
```

Complex vector properties

These are the properties supported by a complex vector instance:

`amax`	Gets the maximum of the absolute values.
`amin`	Gets the minimum of the absolute values.
`distinct`	Gets a new vector with the unique values from the original.
`fft`	Gets the Fast Fourier Transform of the values.
`first`	Gets the first item in the vector.
`imag`	Gets the imaginary components as a vector.
`last`	Gets the last item in the vector.
`length`	Gets the number of values in the vector.
`magnitudes`	Gets magnitudes as a vector.
`mean`	Gets the average of the values in the vector.
`norm`	Gets the Pythagorean norm of the vector.
`phases`	Gets phases as a vector.
`plot`	Shows the vector in a chart.
`prod`	Multiplies all items in the vector.

`real`	Gets the real components as a vector.
`reverse`	Creates a new vector with items in reverse order.
`sqr`	Gets the scalar product of the vector with itself.
`sum`	Gets the sum of all values.

Complex vector methods

These are the methods supported by a complex vector instance:

`all`	Checks if all items satisfy a lambda predicate.
`any`	Checks if exists an item satisfying a lambda predicate.
`filter`	Filters items by value.
`find`	Like `filter` but returns a sequence of the indexes.
`indexOf`	Returns the first index where a value is stored.
`map`	Point-wise transformation of the items in a vector.
`mapReal`	Point-wise transformation of the items in a vector. Returns a real vector.
`reduce`	Reduces all items in a vector to a single value.
`zip`	Combines two vectors using a lambda function.

Complex vector operators

Complex vectors support the same operators as real-valued operators, except for the outer product `^`. On the other hand, they add support for complex vector conjugation using a unary suffix operator:

`'`	Unary suffix operator for complex vector conjugation.

Complex vector conjugation inverts the sign of each imaginary component in the vector. The inner product of two complex vectors conjugates the second vector operand.

Most of the optimisations allowed for real vectors are also available for complex vectors.

Integer vectors

Integer vectors are also supported, using the `ivec` class.

`ivec::new`	Overloaded constructor.
`ivec::ones`	Creates a vector filled with ones.
`ivec::random`	Creates a vector with random values from a uniform distribution.

`ivec::random` has three overloaded variants:

```
-- Ten items. Values between 0 and int.MaxValue - 1.
ivec::random(10);
-- Values between 0 and 999.
ivec::random(10, 1000);
-- Values between -10 and 9.
ivec::random(-10, 10)
```

Integer vector literals can be created using this notation:

```
-- Integer vector creation.
let v1 = [int: 1, 2, 3, 4];
-- Integer vector concatenation.
[int: v1, 5, v1.reverse];
```

Integer vector properties

Integer vectors support these properties:

`abs`	Gets a new vector with absolute values.
`distinct`	Gets a new vector with the unique values from the original.
`first`	Gets the first item in the vector.
`last`	Gets the last item in the vector.
`length`	Gets the number of values in the vector.
`max`	Gets the maximum value from the vector.
`min`	Gets the minimum values from the vector.
`prod`	Multiplies all items in the vector.
`reverse`	Creates a new vector with items in reverse order.
`sort`	Sorts the vector in ascending order.
`sortDesc`	Sorts the vector in descending order.
`stats`	Gets all statistics in one call.

`sum`	Gets the sum of all values.
`toVector`	Converts the integer vector into a double vector.

Integer vector methods

These are the methods for integer vectors:

`all`	Checks if all items satisfy a lambda predicate.
`any`	Checks if exists an item satisfying a lambda predicate.
`filter`	Filters items by value.
`find`	Like `filter` but returns a sequence with the indexes.
`map`	Point-wise transformation of the items in another integer vector.
`mapReal`	Point-wise transformation of the items into a real vector.
`reduce`	Reduces all items in a vector to a single integer value.
`zip`	Combines two integer vectors using a lambda function.

Date vectors

Date vectors are implemented by the `dvec` class. They are mainly intended to support the arguments vector from time series.

`dvec::new`	Overloaded constructor.
`dvec::random`	Creates a vector with random values from a uniform distribution, given an interval of dates. Vector items are not sorted.

Date vector literals can be created using this notation:

```
-- Date vector creation.
let v1 = [date: jan2024, feb2024, 4@jul24];
-- The type indicator can be omited.
let v1 = [jan2024, feb2024, 4@jul24];
-- Date vector concatenation.
[date: v1, 25@dec24, v1.reverse];
```

Date vector properties

Date vectors have fewer properties than numerical vectors, just because dates have fewer operations than numbers. For instance, it makes no sense to add all dates in a vector.

`distinct`	Gets a new vector with the unique values from the original.
`first`	Gets the first item in the vector.
`last`	Gets the last item in the vector.
`length`	Gets the number of values in the vector.
`max`	Gets the maximum value from the vector.
`min`	Gets the minimum values from the vector.
`reverse`	Creates a new vector with items in reverse order.
`sort`	Sorts the vector in ascending order.
`sortDesc`	Sorts the vector in descending order.

Date vector methods

These are the methods supported by date vectors:

`all`	Checks if all items satisfy a lambda predicate.
`any`	Checks if exists an item satisfying a lambda predicate.
`filter`	Filters items by value.
`find`	Like `filter` but returns a sequence with the indexes.
`map`	Point-wise transformation of the items in another date vector.
`reduce`	Reduces all items in a vector to a single date value.
`zip`	Combines two date vectors using a lambda function.

Date vector operators

Date vectors operators match the corresponding date operators:

`+`	Adds a date vector and an integer vector and returns a vector of date. The second operand can also be an integer scalar.

-	Subtracts two date vectors, returning an integer vector with the difference in days. It can also subtract an integer vector from a date vector to return another vector of dates. In both cases, the second operand can be a scalar.

Indexing and slicing

Individual values from vectors are accessed using its position, starting from zero, inside brackets:

```
vec[0];
vec[vec.length - 1]
```

A segment or slice can be extracted as another vector by using this notation:

```
vec[1..vec.length - 1]
```

The above expression removes the first and the last element from a vector. The upper bound is excluded.

The caret (^) can be used in indexes and segments, to count positions from the end. For instance, this expression returns the next to last item of a vector:

```
vec[^1]
```

These equalities hold:

```
vec[1..^1] = vec[1..vec.length - 1];
vec[^5..^2].length = 3
```

Vectors and series also support safe indexers. With normal indexers, like `v[1000]`, an out-of-range reference throws an exception and interrupts the evaluation of the formula. If braces are used instead of brackets, and out-of-range reference returns 0.0 and it is considered as a valid use. For instance, the following expression returns zero:

```
[1, 2, 3, 4]{1000}
```

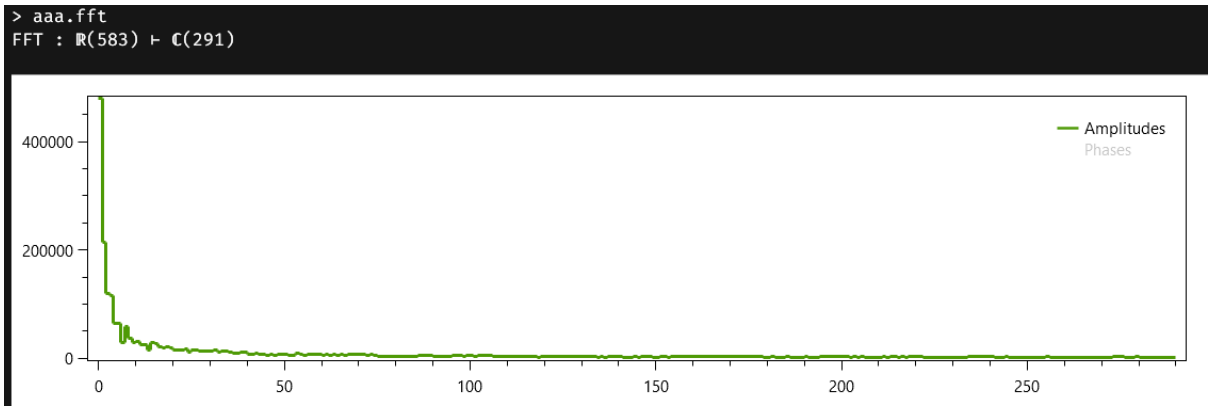
Safe indexers are useful when used inside lambda functions. This expression creates a vector holding the first 30 Fibonacci numbers:

```
vec(30, (i, v) => max(1, v{i-1} + v{i-2}))
```

The Fast Fourier Transform

AUSTRA provides a Discrete Fourier Transform for real and complex vectors, sequences, and series. The core result of the transformation, which is implemented by the `fft` property is a complex vector, but this vector is commonly wrapped inside a `FftModel` class instance, which provides additional helping methods and properties and, among them, a method for inverting the transform and getting back the original samples.

The other function of `FftModel` is to act a semantic marker on behalf of any application that is using the AUSTRA parser. When you execute something like `aaa.fft` in the Austra Desktop application, where `aaa` is a time series, you get a special view for the Fast Fourier Transform based on the returned `FftModel`:



The first line tells us we are seeing a Fast Fourier Transform that has transformed 583 real samples into a complex spectrum containing 291 complex values. Since there is not a pretty and effective way to draw those complex values, the following chart shows the amplitudes of those values and gives us the option to also see their phases.

If we immediately type and execute `ans.inverse`, we will get not the original series, because the date arguments have been discarded by the FFT, but a vector with the original values or coordinates of the time series. The key in the reconstruction is that the `FftModel` keeps track of the fact that the FFT was created from a vector of reals instead of from a vector of complex numbers.

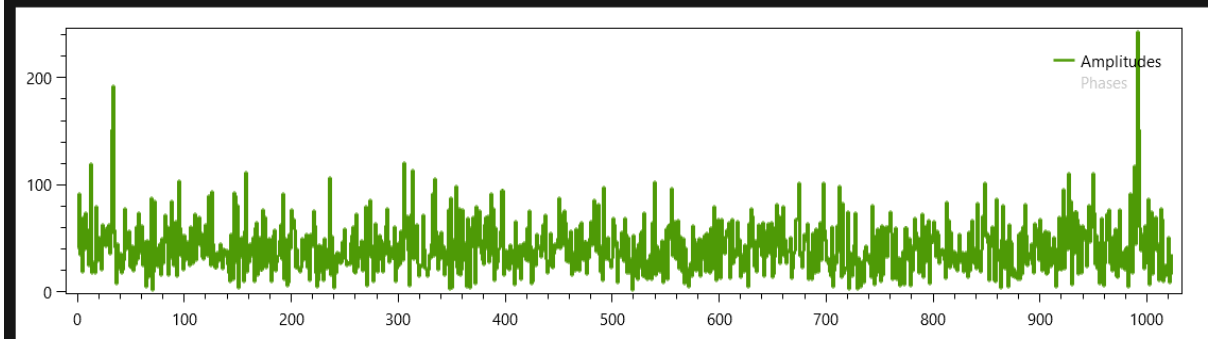
Let us check now how our FFT handles a complex vector. For making things a little more interesting this time, we are going to start with this formula:

```
(cvec::nrandom(1024) + cvec(1024, i => 0.6*sin(i*0.2))).fft
```

There is a noisy component, based on a normal distribution, and then we add a shameless periodic function, affecting just the real part of the complex vector. The first thing we can predict is that the FFT will not have a big zero component: the zero component of any Discrete Fourier Transform is known as the DC, or direct current component, in electronic jargon, because it represents the mean of the samples. Our

new mean will be insignificant because the normal distribution generates both positive and negative terms, and the corresponding plot confirms our suspicions:

```
> (cvec::nrandom(1024) + cvec(1024, i ⇒ 0.6*sin(i*0.2))).fft
FFT : ℂ(1024) ⊢ ℂ(1024)
```



This time, the number of samples in the transform is the same as the number of original samples. It is immediately obvious that there is a periodic component in the samples, which shows as two symmetric peaks at the beginning and end of the spectrum. And, if we immediately execute `ans.inverse`, we get back the original complex vector with all its samples.

FFT properties and indexers

For the sake of clarity, let us group all properties available for FFT models:

`amplitudes`	Gets the amplitudes, or magnitudes, of the transformation numbers.
`inverse`	Performs the inverse transformation for the full spectrum. The algorithm used depends on the kind of source of the transformation.
`length`	Number of samples in the transformation result.
`phases`	Gets the phases of the transformation numbers.
`values`	The full spectrum of the transformation, as a complex vector.

The `FftModel` class also implements an indexer and allows the use of slices and relative indexes.

Sequences

SEQUENCES PROVIDES MOST of the operations from real and complex vectors but avoiding the storage. Sequences are like enumerable types in C# with LINQ and are a requisite for any functional language.

AUSTRA supports three kinds of sequences: `seq`, for real valued sequences, `cseq` for complex ones, and `iseq`, for integer sequences.

Double-valued sequences as light vectors

Let's say we want to calculate factorials. AUSTRA is a functional language, so we don't have explicit loops. We could, however, do this with a vector:

```
vec(10, i => i + 1).prod
```

The above code works fine, but it forces the library to allocate one array of ten items. This is the alternative, using a sequence:

```
seq(2, 10).prod
```

Since the sequence's values are generated only by demand, there's no need for the internal storage.

Sequence constructors

These are the class methods for `seq`:

`seq::new`	Creates a sequence, either from a range, a range, and a step, or from a vector or matrix. See examples below.
`seq::random`	Creates a sequence of random values.
`seq::nrandom`	Creates a sequence of random values, using a normal distribution.
`seq::ar`	Creates a sequence using an autoregressive process.
`seq::ma`	Creates a sequence using a Moving Average process.
`seq::repeat`	Creates a sequence repeating a value.
`seq::unfold`	Generate values from a seed and a generating function.

This code fragment shows some of the available constructors for sequences:

```
seq(1, 10);          -- Numbers from 1 to 10.
seq(10, 1);          -- The inverted sequence.
seq::new(1, 10);     -- ::new was omitted before.
seq(0, 128, τ);      -- A uniform grid with 128 intervals.
seq(v);              -- A sequence from a vector.
seq([sqrt(2), e, π, τ]);
seq(v1^v2);          -- A sequence from a matrix.
seq::random(10);     -- A sequence with 10 random values.
seq::nrandom(10);    -- A sequence with 10 Gaussian random values.
seq::nrandom(10, 2) -- Ten normal samples with variance = 2.
```

Real sequences created with just a lower and an upper bound are always based on integer bounds. For instance, these two expressions represent the same real sequence:

```
seq(1, 10);          -- Numbers from 1 to 10.
seq(1.5, 10.6);      -- Numbers from 1 to 10, too.
```

The reason for this rule is that these sequences have always one unit as their step. It is easier to reason about their behaviour knowing that their boundaries always are integer values.

There are two additional class methods for generating autoregressive, AR(p), and moving average, MA(q), sequences:

```
-- An autoregressive (AR) process of order three.
seq::ar(1000, 1, [0.1, 0.05, 0.01]);
-- A moving average (MA) process of order three.
-- The first term in the vector is the model's mean.
seq::ma(1000, 1, [0, 0.1, 0.05, 0.01])
```

You can materialise the content of a sequence as a vector using the `toVector` property:

```
seq::random(10).toVector
```

The unfold sequence generator

Another class method for creating sequences is `seq::unfold`, which has three variants:

```
-- Powers of 2, from 2 to 1024.
seq::unfold(10, 2, x => 2x);
-- Maclaurin series for exp(1).
seq::unfold(100000, 1, (n, x) => x / (n + 1)).sum + 1;
-- Real-valued Fibonacci sequence.
seq::unfold(50, 1, 1, (x, y) => x + y);
```

The `unfold` sequence generator is important for the language since it can express iterative behaviour in a language with no explicit loops and conditionals. The alternative to iteration is recursion, which is also provided by AUSTRA, but a recursive function is almost always more expensive, even in the presence of tail optimisations.

Let us consider, for instance, how we would write a function for the greatest common divisor. When learning about function definitions, we saw that we could define a recursive function for this task:

```
-- The recursive version
def gcd1:"Recursive GCD"(a, b: int): int =
    let rm = a % b in
        iff(rm = 0, b, mcd(b, rm))
```

This is a fine recursive definition that even is amenable to tail recursion optimisation. Now compare with the iterative alternative:

```
-- The iterative version
def gcd2:"Iterative GCD"(a, b: int): int =
    iseq::unfold(10000, a, b, (x, y) => x % y).while(x => x > 0).last
```

Even though it looks more verbose, the second definition is almost twice faster than the recursive definition. Note, however, that we have used the class `iseq` instead of `seq`, so the property `last` directly returned an integer value. We could have kept `seq` by simply adding `toInt` after calling `last`:

```
def gcd3:"Iterative GCD"(a, b: int): int =
    seq::unfold(10000, a, b, (x, y) => x % y)
    .while(x => x > 0).last.toInt
```

Still, the advantage of the iterative definition will be huge compared to the recursive definition.

Note

Two things to consider:

- If we had not included the while method, every call to that unfold sequence would end in failure as soon as the algorithm would have tried to divide by zero. The while method avoids that problem and gives us the value we need.
- The unfold generator requires a first parameter stating how many values to create. We could have devised a variant of unfold for creating an infinite-length sequence. I have preferred, however, to put the burden of picking a high enough value on your shoulders. It is security against convenience.

Methods and properties

These are the properties supported by all sequences of real values:

`acf`	The Autocorrelation function.
`distinct`	Select unique values, with no predefined order.
`fft`	Calculates a Fast Fourier Transform.
`first`	Gets the first term of the sequence.
`last`	Gets the last term of the sequence.
`length`	Gets the number of elements in the sequence.
`max`	Get the maximum value in the sequence.
`min`	Get the minimum value in the sequence.
`pacf`	The Partial Autocorrelation function.
`plot`	Plots the sequence.
`prod`	Multiplies all values in the sequence.
`sort`	Sorts values in ascending order.
`sortDesc`	Sorts values in descending order.
`stats`	Gets all statistic moments of the sequence.
`sum`	Sums all values in the sequence.
`toVector`	Materialises the sequence into a vector.

These are the methods that can be used with a sequence of real values:

`all`	Checks if all items in the sequence satisfy a predicate.
`any`	Checks if there is an item in the sequence satisfying a predicate.
`ar`	Estimates coefficients for an AR(p) model.
`arModel`	Creates a full AR(p) model.
`filter`	Returns items of the original sequence satisfying a predicate.
`ma`	Estimates coefficients for an MA(q) model.
`maModel`	Creates a full MA(q) model.

`map`	Transforms items with the help of a lambda function.
`reduce`	Conflates all values in a sequence using a lambda.
`until`	Returns a prefix of a sequence until a value satisfying a predicate is found.
`while`	Returns a prefix of a sequence while values satisfy a predicate.
`zip`	Combines two sequences using a lambda function.

As seen before, the `while` method excludes from its returning sequence the first value satisfying its predicate. On the contrary, `until` includes that very value in the returning sequence.

Sequence operators

Sequence operators mimics most of vector's operators.

```
seq(1, 10) * seq(10, 1)  -- The dot product.
```

For instance, simple operators can be used to change the underlying distribution of a random sequence.

```
seq::random(100) * 2 - 1;
-- Check the moments of the above distribution.
(seq::random(100) * 2 - 1).stats
```

Note

Unary operators for sequences could, in theory, be implemented using `map`, and binary operators can also be written using `zip`.

However, in most cases, having an explicit operator results in a faster implementation. It is most evident for sequences backed by a vector, but it also happens for other kinds of sequences. For instance, when a range or grid sequence is negated, you can implement the result using another range or grid sequence.

Integer sequences

Integer sequences are represented by the `iseq` class.

Class methods

These are the class methods supported by `iseq`:

`iseq::new`	Creates a sequence, either from a range, a range, and a step, or from an integer vector.

`iseq::random`	Creates a sequence of random integers. You can pass an upper bound, or an interval for values.
`iseq::unfold`	Like `seq::unfold`, but with integer arguments.

Methods and properties

These properties can be used with integer sequences:

`distinct`	Select unique values, with no predefined order.
`first`	Gets the first term of the sequence.
`last`	Gets the last term of the sequence.
`length`	Gets the number of elements in the sequence.
`max`	Get the maximum value in the sequence.
`min`	Get the minimum value in the sequence.
`plot`	Plots the sequence.
`prod`	Multiplies all values in the sequence.
`sort`	Sorts values in ascending order.
`sortDesc`	Sorts values in descending order.
`stats`	Gets all statistic moments of the sequence.
`sum`	Sums all values in the sequence.
`toVector`	Materialises the sequence into a vector.

These are the available methods:

`all`	Checks if all items in the sequence satisfy a predicate.
`any`	Checks if any item in the sequence satisfies a predicate.
`filter`	Returns items of the original sequence satisfying a predicate.
`map`	Transforms items with the help of a lambda function.
`mapReal`	Transforms items with the help of a lambda function.
`reduce`	Conflates all values in a sequence using a lambda.

`until`	Returns a prefix of a sequence until a value satisfying a predicate is found.
`while`	Returns a prefix of a sequence while values satisfy a predicate.
`zip`	Combines two sequences using a lambda function.

This example shows how to calculate the Collatz sequence using integer sequences:

```
let collatz(n: int) =
    iseq::unfold(1000000, n, x => iff(x.even, x / 2, 3x + 1))
    .until(x => x = 1);
collatz(137)
```

Though the generator is created with a big enough upper limit, the sequence stops when a `1` is generated. The `until` method can also be written this way:

```
let collatz(n: int) =
    iseq::unfold(1000000, n, x => iff(x.even, x / 2, 3x + 1))
    .until(1)
```

Complex sequences

Complex sequences can also be used, with the `cseq` class.

Class methods

These are the class methods supported by `cseq`:

`cseq::new`	Creates a sequence, either from a complex interval, or from a complex vector.
`cseq::random`	Creates a sequence of random values from a uniform distribution.
`cseq::nrandom`	Creates a sequence of random values with a standard normal distribution.
`cseq::unfold`	Like `seq::unfold`, but with complex arguments.

Methods and properties

These properties can be used with complex sequences:

`distinct`	Select unique values, with no predefined order.
`first`	Gets the first term of the sequence.

`last`	Gets the last term of the sequence.
`length`	Gets the number of elements in the sequence.
`plot`	Plots the sequence.
`prod`	Multiplies all values in the sequence.
`sum`	Sums all values in the sequence.
`toVector`	Materialises the sequence into a complex vector.

And these are the available methods:

`all`	Checks if all items in the sequence satisfy a predicate.
`any`	Checks if any item in the sequence satisfies a predicate.
`filter`	Returns items of the original sequence satisfying a predicate.
`map`	Transforms items with the help of a lambda function.
`mapReal`	Transforms items with the help of a lambda function.
`reduce`	Conflates all values in a sequence using a lambda.
`until`	Returns a prefix of a sequence until a value satisfying a predicate is found.
`while`	Returns a prefix of a sequence while values satisfy a predicate.
`zip`	Combines two sequences using a lambda function.

Delayed execution

Sequences are modelled after .NET LINQ enumerable interfaces, and so many other functional libraries. One of the most interesting features of these libraries is *delayed execution*.

Applying a method or an operator on a sequence does not means that it will automatically scan the sequence values. Let's start with a simple example:

```
-seq(1, 1000)
```

The above code first creates a sequence that will enumerate numbers from 1 to 1000. Creating the sequence means creating a small instance of an internal class that can be called later to yield the values in the sequence. The unary minus, however, takes

that sequence generator and returns another generator that yields values in descending order from the interval $[-10, -1]$. It does not yet force the sequence enumeration. Enumeration takes place as the last operation, as you hit F5 on the AUSTRA desktop, as the application needs to print the values created by the expression. The same would happen with this expression, which plots the sequence as a series:

```
(-seq(1, 1000)).plot
```

It is the `plot` method the trigger which starts the internal loop for generating all the values. You could even intercalate another method call before the plot, without triggering enumeration:

```
-- Sort the negated values in ascending order.
(-seq(1, 1000)).sort.plot;
-- Square values, select multiples of three and sort descending.
seq(1, 1000).map(x => x ^ 2).filter(x => x % 3 = 0).sortDesc.plot;
-- Methods like sum, prod, any,or first can also trigger evaluation.
seq(1, 100).filter(x => x % 2 = 0).map(x => x ^ 2).sum
```

Matrices

AUSTRA MATRICES ARE represented by the `matrix` class. They are implemented as row-first, double precision dense matrices.

The AUSTRA `matrix` class is based on three different C# structures: `Matrix`, `LMatrix`, and `RMatrix`. The compiler takes automatically care of any conversions when needed.

Matrix construction

A matrix can be constructed by enclosing its components inside brackets:

```
[1, 2, 3; 2, 3, 4; 3, 4, 5]
```

Rows must be separated by semicolons (`;`), and items in a row must be separated by commas. This syntax does not allow writing a matrix with only one row, since the compiler would not be able to tell it from a vector. A workaround is to write a matrix with only one column and transpose it:

```
[1; 2; 3; 4]'
```

You can also create a new matrix by concatenating two existing matrices, or a matrix and a vector. You can use either vertical or horizontal concatenation:

```
let m = [1, 2; 3, 4], v = [1, 1];
-- Horizontal concatenation (2x4 matrix).
[m, m];
-- Horizontal concatenation (2x6 matrix).
[m, m, m];
-- Horizontal concatenation (2x3 matrix).
[m, v];
[v, m];
-- Vertical concatenation (4x2 matrix).
[m; m];
-- Vertical concatenation (6x2 matrix).
[m; m; m];
-- Vertical concatenation (3x2 matrix).
-- The vector is handled as a row vector.
[m; v];
[v; m];
```

Class methods

These class methods are available for creating matrices:

`matrix::new`	Overloaded constructor (see below).
`matrix::rows`	Creates a matrix given its rows as vectors.
`matrix::cols`	Creates a matrix given its cols as vectors.
`matrix::eye`	Creates an identity matrix given its size.
`matrix::diag`	Creates a diagonal matrix given the diagonal as a vector.
`matrix::random`	Creates a matrix with random values from a uniform distribution.
`matrix::nrandom`	Creates a matrix with random values from a normal standard distribution.
`matrix::lrandom`	Creates a lower-triangular matrix with random values from a uniform distribution.
`matrix::lnrandom`	Creates a lower-triangular matrix with random values from a standard normal distribution.
`matrix::cov`	Creates a covariance matrix given a list of series.
`matrix::corr`	Creates a correlation matrix given a list of series.

Methods and properties

These are the properties available for matrices:

`amax`	Gets the absolute maximum.
`amin`	Gets the absolute minimum.
`chol`	Calculates the matrix of the Cholesky factorisation.
`cholesky`	Calculates the full Cholesky factorisation.
`cols`	Gets the number of columns.
`det`	Calculates the determinant.
`diag`	Gets the main diagonal as a vector.
`evd`	Calculates the Eigenvalues Decomposition.
`inverse`	Gets the inverse of this matrix.

`isSymmetric`	Verifies if the matrix is a symmetric one.
`max`	Gets the maximum value from the cells.
`min`	Gets the minimum value from the cells.
`rows`	Gets the number of rows.
`stats`	Returns statistics on cells.
`sum`	Gets the sum of all values.
`trace`	Gets the sum of the main diagonal.

And these are the supported methods:

`all`	Checks if all cells satisfy a lambda predicate.
`any`	Checks if exists a cell satisfying a lambda predicate.
`getCol`	Gets a column by its index.
`getRow`	Gets a row by its index.
`map`	Creates a new matrix with transformed cells.
`redim`	Creates a new matrix with a new size.

Matrix operators

These are the operators available for matrices:

`+`	Adds two matrices, or a matrix and a scalar.
`-`	Subtracts two matrices, or a matrix and a scalar. It is also used as a unary operator.
`*`	Matrix * Matrix = Matrix multiplication. Matrix * Number = Matrix scaling. Matrix * Vector = Vector transformation. Matrix * Complex vector = Complex vector transformation. Vector * Matrix = Vector is transposed and then transformed. Complex vector * Matrix = Vector is transposed and then transformed.
`.*`	Point-wise multiplication of two matrices.
`./`	Point-wise quotient of two matrices.

`/`	Divides a matrix by a scalar, but also divides either a vector or a matrix by a matrix, for solving linear equations.
`'`	Unary suffix operator for matrix transpose.

These examples show how to solve linear equations for a vector, using division by a matrix:

```
let m = matrix::random(5) + 0.01,
    v = vec::random(5),
    answer = v / m in
    m * answer - v
```

Solving equations for a matrix is also possible:

```
let m = matrix::random(5) + 1;
matrix::eye(5) / m - m.inverse
```

Internally, the LU factorisation of the matrix is used for equation solving, for the general case. When the matrix at the left is a triangular matrix, a most efficient algorithm is used.

Optimisations

The compiler performs some optimisations for matrix operations. For instance, these two expressions yield the same result, but the second one avoids one matrix transpose:

```
let m = matrix::random(10), v = vec::random(10);
-- Transpose a matrix and then transform a vector:
m' * v;
-- Changing the order of operands avoids a transpose:
v * m;
```

The compiler also detects when the second matrix in a matrix multiplication is being transposed:

```
let m1 = matrix::random(10), m2 = matrix::random(10) in
    m1 * m2'
```

This pattern is handled by the `MultiplyTranspose` method, which not only saves the time spent in the transpose but also avoids a temporal memory allocation.

These two operation patterns are also detected and implemented with a single method call:

```
let m = matrix::random(10);
let v1 = vec::random(10);
let v2 = vec::random(10);
let scaleFactor = 0.1;
```

```
m * v1 ± v2;
m * v1 ± scaleFactor * v2;
```

These special operations are implemented by the `MultiplyAdd` and `MultiplySubtract` group of overloaded methods.

Indexing and slicing

Individual cells are accessed using the row and column inside brackets:

```
mat[0, 0];
mat[mat.rows - 1, mat.cols - 1]
```

All indexes start from zero. If the row index is omitted, a whole column is returned:

```
mat[, 0]
```

Omitting the column number yields a whole row:

```
mat[0,];
mat[0]
```

Carets can also be used in any of the two indexes, to count positions from the end. For instance, this expression returns the rightmost lower cell of the matrix:

```
mat[^1, ^1]
```

Columns and rows can also be extracted as vectors using relative indexes:

```
mat[, ^2]; -- Next to last column.
mat[^2,]   -- Next to last row.
```

Ranges are accepted for both dimensions, and can be combined with indexes too:

```
-- Remove last row and last column.
mat[0..^1, 0..^1];
-- Last row without first and last items.
mat[^1, 1..^1]
```

Eigenvalues Decomposition

An *eigenvalue* λ and its associated *eigenvector* v are any pair of values that satisfy this equation for a square matrix M:

$$Mv = \lambda v$$

It means that, when the matrix transforms an eigenvector, the result is the same vector, except for a scale factor.

The Eigenvalue Decomposition of a matrix is an algorithm that identifies all the pairs of eigenvalues and eigenvector for a given square matrix. You can efficiently find eigenvalues and eigenvectors in AUSTRA applying the `evd` method on a matrix:

```
let m = matrix::random(10);
let e = mat.evd;
-- This is a matrix with all eigenvectors as columns.
e.vectors;
-- This is a complex vectors with all eigenvalues.
e.values;
-- An alternative representation of eigenvalues, using a real matrix.
e.d
```

A real matrix can have both real and complex eigenvalues, and that is why the `values` property of the decomposition is a complex vector. When there is a complex eigenvalue, its complex conjugate is also an eigenvalue of the matrix. Aside from `values`, the eigenvalues are also returned in a `d` property, which is a real block diagonal matrix. Each real eigenvalue is placed in the main diagonal, and complex eigenvalues are represented as 2x2 blocks in the diagonal.

The best way to visualise how `evd` returns eigenvalues is to show an example. These are the eigenvalues of a 4x4 random matrix:

```
> e.values
ans ∈ ℂ(4)
<2.02631; 0>
<-0.229546; 0.093745>
<-0.229546; -0.093745>
<0.364586; 0>

> e.d
ans ∈ ℝ(4×4)
2.02631          0          0          0
      0  -0.229546   0.093745          0
      0  -0.093745  -0.229546          0
      0          0          0   0.364586
```

The second and third eigenvalues are a conjugated pair of complex numbers. See how they are represented in the block diagonal matrix, using four cells.

When m is a square matrix, the following mathematical equivalence must hold:

$$m * m.evd.vectors = m.evd.vectors * m.evd.d$$

In practice, however, we must take the loss of precision into account. For verifying an EVD, you can use this formula in AUSTRA:

```
let m = matrix::random(32), e = m.evd in
  (m * e.vectors - e.vectors * e.d).amax <= 1e-12;
let lm = matrix::lrandom(32), m = lm * lm', e = m.evd in
  (m * e.vectors - e.vectors * e.d).amax <= 1e-12;
```

Symmetric matrices, as the one generated for the second example above, are decomposed using a more efficient algorithm, so Austra checks symmetry first, before applying any of these algorithms.

LU Factorisation

The LU (lower-upper) factorisation algorithm takes a square matrix and generates a lower-triangular matrix L and an upper-triangular matrix U that, when multiplied, regenerates the original matrix. The algorithm may also reorder rows for the sake of numerical stability.

The `lu` property, when applied to a square matrix, returns a `LU` structure from the Austra library that provides these properties:

`det`	Gets the determinant of the original matrix.
`lower`	Gets the lower-triangular matrix from the factorisation.
`perm`	Gets an integer vector with the permutations.
`size`	Gets the number of rows/columns from the original matrix.
`upper`	Gets the upper-triangular matrix from the factorisation.

More relevant for us are the two overloads of the `solve` method of this structure:

`solve(vec)` `solve(matrix)`	Solves the equations $Mx = v$ or $Mx = m$, where M is the factorised matrix, v is a vector and m is another matrix.

The `solve` method from the original matrix does the same work, but it delegates the solution to a LU factorisation created on the fly. If we need to solve more than one equation involving the same matrix at the left side, it is more efficient to perform the LU factorisation once, and reuse the result for each linear system, as this code shows:

```
-- This a matrix whose LU factorisation will be reused.
let m = matrix::random(10);
-- The LU factorisation is computed here.
let lu = m.lu;
-- Now we generate a vector and a matrix for the right sides.
let n = matrix::random(m.rows), v = vec::random(m.rows);
-- Solve m*x=n and m*y=v, and check the accuracy of the results.
(m * lu.solve(n) - n).amax;
(m * lu.solve(v) - v).amax;
```

The accuracy, for the above example, is near `1e-16` or `1e-15`.

Cholesky decomposition

Another important factorisation is the Cholesky decomposition. It requires a square matrix, but this time, the matrix must be a symmetric one. The Cholesky algorithm finds, for a given M matrix, a lower-triangular matrix C such that:

$$M = C \cdot C'$$

Of course, matrices generated by multiplying a lower-triangular matrix by its transpose are symmetric, as is easy to demonstrate. Not only that: the resulting matrix must be a positive-definite matrix, with its determinant greater than zero since the determinant of a matrix product is the product of the determinants. C, when exists, can be considered a sort of square root of the original matrix M.

A matrix provides two properties related to the Cholesky decomposition. The `chol` property returns the lower-triangular matrix when it exists or throws an exception otherwise. The `cholesky` property, on the other hand, returns an object that encapsulates the Cholesky matrix. The reason for this apparent detour is that the returned object implements these two overloads of a `solve` method:

`solve(vec)` `solve(matrix)`	Solves the equations $Mx = v$ or $Mx = m$, where M is the factorised matrix, v is a vector and m is another matrix.

The lower-triangular matrix computed by the decomposition can also be retrieved from the object return by `cholesky` using its `lower` property.

List comprehensions

A LIST COMPREHENSION IS a syntactic sugar construct for filtering and mapping sequences, vectors, and series. They simplify writing lambda functions for methods, and they are easier to read and understand.

Syntax

Suppose you need to write a formula like this one:

```
seq(1, 100).filter(x => x.odd).map(x => x²)
```

This code is not a candidate for the Turing Award: it takes the squares of all odd numbers between 0 and 100. You had to type two lambda functions, including arrows and lambda parameters, and you also had to explicitly mention the `filter` and the `map` methods, including the parentheses enclosing their arguments.

This alternative expression does the same, is shorter to type and easier to read:

```
[x <- seq(1, 100) : x.odd => x²]
```

With this trick, we have avoided repeating the declaration of the parameter `x` in the two lambda definitions used in the expression.

In this example, since the source of all numbers is a range sequence, you could also use a simpler expression for the range:

```
[x <- 1..100 : x.odd => x²]
```

The syntax for this construct can be summarised like this:

```
[identifier <- generator : filter => mapping]
```

Both `filter` and `map` are optional:

```
-- This expression...
[x <- 1..100];
-- ... is equivalent to this one:
seq(1, 100)
```

Types in list comprehensions

The type assigned to the whole list comprehension expression is the same of its generator. You can keep applying methods or operators to the result:

```
[x <- 1..100 : x % 2 = 1 => x^2].sortDesc;
[x <- 1..100] .* ([x <- 1..100] + 1)
```

Special care is needed when the generator is a time series, because the identifier in the head of the list comprehension is typed as `double` in the mapping section, but it is a `Point<Date>` in the filter section:

```
let mean = msft.mean in
  [x <- msft : x.date >= jan2015 => x - mean]
```

Generators

As we have seen, range expressions can be used as generators. We support four variants of range expressions inside list comprehensions:

```
-- Equivalent to iseq(1, 100)
[x <- 1..100];
-- Equivalent to seq(1, 100)
[x <- 1.0..100.0];
-- Even integers from 0 to 100.
[x <- 0..2..100];
-- The same as seq(0, 1024, 2 * pi)).
[x <- 0..1024..2pi];
```

In the last example, only the upper bound is real, so the compiler handles the generator as a real sequence.

The parameter identifier and the membership operator can also be drop, and the above examples simplify this way:

```
[1..100];
[1.0..100.0];
[0..2..100];
[0..1024..2pi];
```

We have mostly used constants for the range generators so far but, of course, each part of the generator could be an expression:

```
[x <- pi - 1..32 * 32..sqrt(200)];
```

Quantifiers in list comprehensions

Logical quantifiers can be used at the beginning of a list comprehension. The allowed quantifiers are `all` and `any`, as the corresponding methods in vectors and sequences. They are not keywords, but when used at the beginning of a list comprehension, they are considered *contextual keywords* for syntax highlighting.

This is a very simple example of a quantifier in a list comprehension expression and its equivalent form using methods:

```
[any x <- 10..100 : x * x = x + x];
iseq(10, 100).any(x => x * x = x + x)
```

Both expressions are compiled as Boolean expressions. Note that a qualified list comprehension does not allow a mapping section.

The quantified list comprehension is marginally shorter than a call to `any` or `all`. Why, then, do we bother supporting this syntax? The reason is that we can embed a qualified predicate inside a normal list comprehension:

```
-- Find all prime numbers between 2 and 100:
[x <- 2..100 : all div <- 2 .. x - 1 : x % div != 0];
-- Equivalent, but longer:
iseq(2, 100).filter(x => iseq(2, x - 1).all(div => x % div != 0))
```

We need no inner brackets inside the main list comprehension since it is evident how the qualified condition is nested. We could even add a mapping at the end of the comprehension to transform the calculated prime numbers, if required.

If we use regular lambdas, we will be nesting a lambda definition inside another. The generated code for the list comprehension also uses nested lambdas, but with easier- to-understand syntax. The inner lambda is "capturing" the parameter of the outer lambda, so we must be careful when naming local variables.

The mathematical symbols ∀ and ∃ are also accepted as synonyms of `all` and `any`:

```
[∃x <- 10..100 : x * x = x + x];
[x ∈ 2..100 : ∀y ∈ 2..x - 1 : x % y != 0];
```

These symbols can be typed by pressing `CtrlQ+A` or `CtrlQ+E` in the Code Editor.

Splines

SPLINES ARE PIECEWISE defined functions using cubic polynomials for interpolating or smoothing curves. Austra can create splines for time series, using dates as arguments, or for any pair of vectors containing abscissas and coordinates, respectively. There is also a shortcut for creating this second kind of splines given a grid on an interval and an arbitrary function.

Creating splines

All spline kinds are created using overloaded variants of the same class method:

`spline::new`	Creates a spline either from a series, a couple of vectors, or a grid and a lambda function.

This example shows how to create and use a spline based on a time series:

```
let s = spline(appl) in
    s[appl.last.date - 15]
```

The example creates a spline based on the series values, and then the spline is used to interpolate the value fifteen days before the last date stored in the series.

At a first glance, it may seem than interpolating a daily series does not make sense, since AUSTRA dates do not include a time fraction. Nevertheless:

- Even daily series have gaps corresponding to holidays.
- You can still use a real value for interpolating a spline with date arguments.

The following formula, for example, finds what would be the value at a middle time between two consecutive dates:

```
let s = spline(appl) in
    s[4@jul20.toInt + 0.5]
```

We are adding half of a day, i.e., twelve hours, to the numerical equivalent of a date, if the stored values in the time series corresponds to each day's midnight.

Splines can also be used to interpolate existing data and functions:

```
-- Use a function over a uniform grid.
let s1 = spline(0, τ, 1024, cos);
s1[π/4] - sqrt(0.5);
s1.derivative(π/4);
```

```
-- Use two arbitrary vectors with the same length.
let s2 = spline([1, 3, 4, 5], [0, 1, 0.8, 0]);
s2[2]
```

Indexers, methods, and properties

All splines have these four properties:

`area`	The total area below the spline.
`first`	The lower bound for the abscissas. It is a date for splines based on series, and a double value, otherwise.
`last`	The upper bound for the abscissas. It is a date for splines based on series, and a double value, otherwise.
`length`	Gets the number of polynomials in the spline.

For instance, we can use it to approximate the area below a normal distribution:

```
-- The integral over a reasonable interval.
spline(-10, 10, 10000, x => exp(-x²)).area;
-- The expected result.
sqrt(π)
```

Note

When `area` is used on a series-based spline, dates are automatically interpreted as real values, so a day is equal to the unit value.

These are the methods implemented by splines:

`derivative`	Calculates the smoothed derivative at a given point of the spline range.
`poly`	Gets the cubic polynomial at a given index in the spline.

The `poly` method has two overloads: one receives an integer, and the other allows a C# `Index` as its argument:

```
-- Let's define a spline with a function over a uniform grid.
let s1 = spline(0, τ, 1024, cos);
-- Retrieve the polynomial for the first segment of the spline.
s1.poly(0);
-- Two alternatives for retrieving the last polynomial:
s1.poly(s1.length - 1);
s1.poly(^1);
```

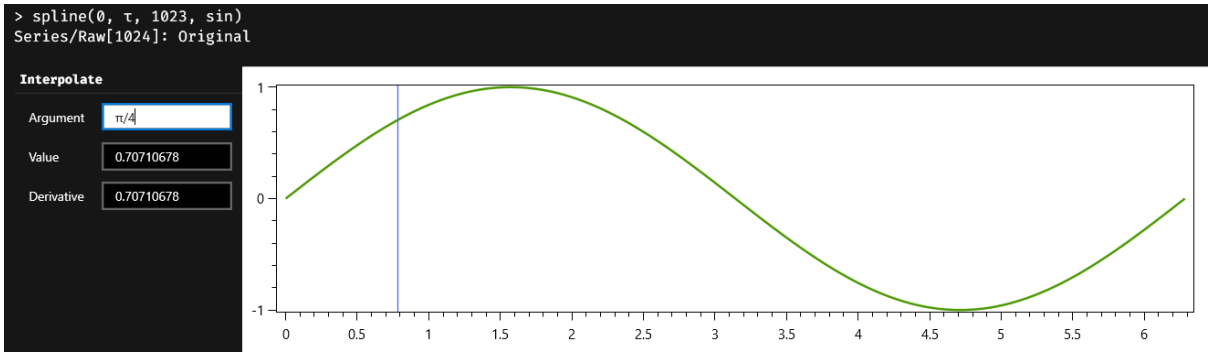
Polynomials retrieved with the `poly` method accepts values in the closed interval [0, 1]. The spline interpolator must find the polynomial, subtract the initial argument

for the corresponding segment and scale the remaining offset according to the length covered by the segment. Each polynomial provides two methods, for evaluating its value and its derivative at a point in the closed interval [0,1], and one property, `area`, for evaluating the definitive interval of the polynomial over its valid interval:

`area`	The definite integral over the interval [0,1].
`eval`	Evaluates the polynomial at the given argument.
`derivative`	Gets the derivative of the polynomial.

Interacting with a spline

When a spline is evaluated in the Austra Desktop application, an interactive control is shown. You can enter values in the `Argument` text box to evaluate the spline and its derivative at the supplied argument. This control appears no matter which argument type is being used for the spline:



For numeric arguments, you can even type an Austra formula in the text box, and Austra evaluates the formula when `Enter` is pressed.

Models

THE MODEL CLASS IS a general-purpose container for algorithms that do not fit well as members of other classes. These models are generally shown by the Austra Desktop application as interactive controls, allowing users to explore the whole range of solutions available for each model.

Mean Variance Optimiser

Mean variance optimisation (MVO) is a mathematical optimisation for maximising the expected return of a portfolio given a level of risk. Inside a polytope, the fundamental algorithm is an optimiser for a quadratic objective function. A *polytope* is just a fancy name for a polyhedron in a high-dimensional space.

The MVO is implemented by the `MvoModel` class from the Austra library, and it is available for the AUSTRA language by executing the `model::mvo` class method.

Let us assume we have a portfolio with three assets, and we want to find the three optimal weights, one for each asset. The most general method overload of the MVO would be like this:

```
-- This example assumes we are dealing with three assets.
model::mvo(
    -- A 3D-vector for returns and a 3x3 covariance matrix.
    retVec, covMatrix,
    -- Two 3D-vectors for lower and upper bounds.
    [0, 0, 0], [1, 1, 1],
    -- A label for easy identification of each asset.
    "Name1",
    "Name2",
    "Name3")
```

- We have purposefully avoided stating any values for the *retVec* and *covMatrix* variables. These two variables would contain a vector with the expected return of each asset and a covariance matrix for these assets.
- It is not obvious how expected returns are to be calculated. Financial series are seldom stationary, so the expected return would normally depend on time.
- The covariance matrix faces a similar problem.
- The lower and upper bounds, on the contrary, are generally easier to set; they are just the minimum and maximum weights we desire for each asset in the portfolio.
- The MVO algorithm automatically adds another condition for the weights: their sum must be equal to one.

For the sake of the example, we are going to assume an arbitrary rentability for each of our three hypothetical assets. For the covariance matrix, we will make things more

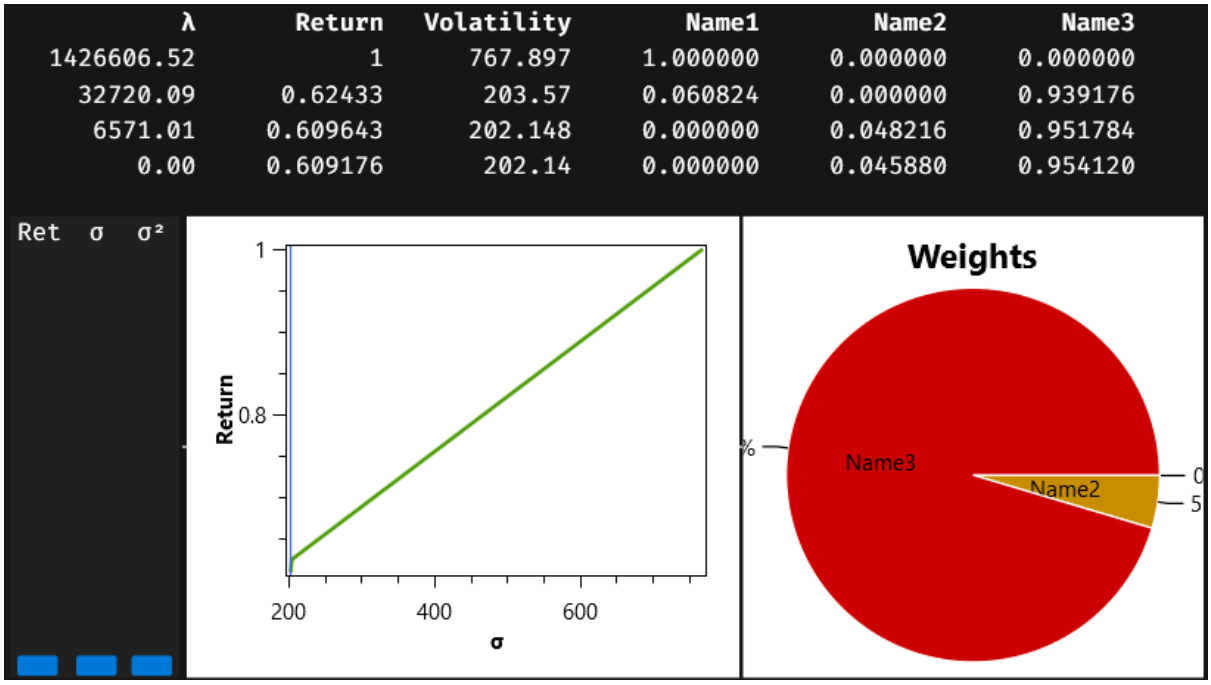
interesting by creating a fake matrix with three of our series examples: aaa, aab and aad:

```
> matrix::cov(aaa, aab, aad)
ans ∈ ℝ(3×3)
 589666   525180  19023.8
 525180   553027  16232.4
19023.8  16232.4  42045.1
```

The main diagonal of the above matrix tells us how volatile each of our assets is. We can see that aaa has the greater variance and that aad has the lesser variance and, consequently, the lower associated risk. So, we will assume that the first asset provides the better return, followed by the second and third assets:

```
model::mvo(
    [1, 0.8, 0.6],
    matrix::cov(aaa, aab, aad),
    [0, 0, 0], [1, 1, 1],
    "Name1", "Name2", "Name3")
```

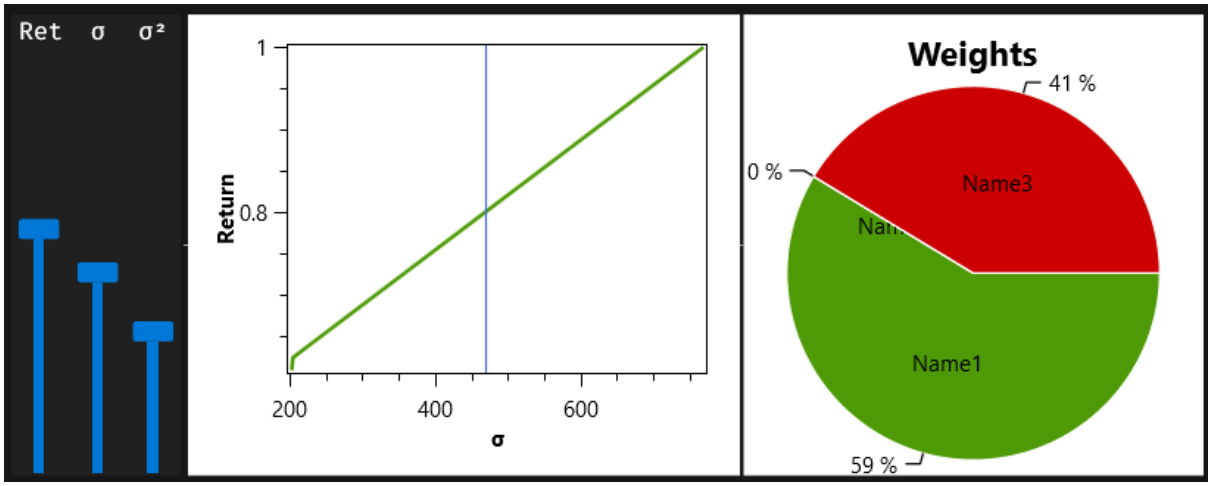
If we execute this code, we will get the following interactive output in the area with results from the Austra Desktop application:



The first part of the output is a table enumerating portfolios from the so-called efficient frontier. There are four such portfolios in our example. The first portfolio maximises the expected return, but it is also the one with more volatility, or risk. This portfolio only includes the first asset; the weight for this asset is one, and the rest of the weights are zero. The last listed portfolio is the one with the lesser volatility and return, and it is a mix of the second and third assets. Each portfolio includes a value for the lambda column, which mathematically is the value of the Lagrange multiplier

for this solution. From the business point of view, lambda is an indicator of the associated risk.

Portfolios in the efficient frontier are important because they represent turning points in the strategy for changing asset weights. Any portfolio interpolated from two portfolios on the efficient frontier is a viable solution to our problem. And that is the mission of the three sliders on the left side of the charts: you can select either a desired return, a standard deviation, or a variance, and the charts will show you which weights are needed for the selected portfolio. This is what we get if we choose an expected return of approximately 0.8:



The required portfolio must be a combination of 59% from the first asset and another 41% from the third asset.

More class method overloads

For our example, we chose arbitrary names for the assets that compose our portfolio. When these assets are related to series in our session, it is easier to use the name of the series for this task:

```
model::mvo(
    [1, 0.8, 0.6],
    matrix::cov(aaa, aab, aad),
    [0, 0, 0], [1, 1, 1],
    aaa, aab, aad)
```

Now, the series variables are mentioned twice in the formula. We could change the formula this way:

```
model::mvo(
    [1, 0.8, 0.6],
    aaa, aab, aad)
```

This is the simplest method overload for the MVO. Note that we have also removed the lower and upper bounds, making the natural assumption that all weights will stay

in the [0,1] interval. We still need, however, to explicitly state the expected returns, but Austra infers that we want to use the covariance matrix for the three used series.

Additional constraints

Optimisation problems frequently include additional constraints beyond the simple limits we have shown so far. As a matter of fact, since we have not included a constraint for the total sum of weights, the `model::mvo` method has automatically added this constraint to the problem:

$$w_{aaa} + w_{aab} + w_{aad} = 1$$

Let's say we want another constraint. For instance, the first asset's weight must always be greater or equal to the third asset's weight:

$$w_{aaa} - w_{aad} \geq 0$$

The most general form for this kind of constraint is a list of equations following this pattern:

$$l_{i,0}w_0 + l_{i,1}w_1 + l_{i,2}w_2 \bowtie r_i$$

Here, $l_{i,j}$ stands for left side and r_i means right side. That strange symbol $\bowtie$ only means that we can substitute it either with an equality or an inequality. So, more generally, our additional constraints could always be written as:

$$L\vec{w} \bowtie \vec{r}$$

L is a matrix with as many columns as assets in the problem and an arbitrary number of rows, $\vec{r}$ is a vector with the same number of items as rows in the left-side matrix. Still, we must find a way to determine which relational operator must be used for each of the constraints.

The `MvoModel` class provides an overloaded method for adding constraints to an already existing model:

```
mvoModel.setConstraints(lhsMatrix, rhsVector, opsIntVector)
```

The first parameter must be a matrix; the second parameter must be a real vector; and the third parameter must be an integer vector. Items in the third parameter are interpreted according to their signs. A positive value means a greater or equal operation; a negative value stands for a lesser or equal relationship; and zero means equality. The third parameter can be omitted when all constraints are equality constraints.

This way, if we want to combine the sum-of-weights constraint with our additional constraint, we will need the following code:

```
model::mvo(
    [1, 0.8, 0.6],
    aaa, aab, aad).setConstraints(
        [1, 1, 1; 1, 0, -1], [1, 0], [int:0, 1])
```

These are the portfolios from the efficient frontier, with the additional constraint:

λ	Return	Volatility	aaa	aab	aad
1426606.52	1	767.897	1.000000	0.000000	0.000000
0.00	0.8	409.194	0.500000	0.000000	0.500000

We could even drop the first constraint because the optimiser will add it when it is not present:

```
model::mvo(
    [1, 0.8, 0.6],
    aaa, aab, aad).setConstraints(
        [1; 0; -1]', [0], [int:1])
```

Please note the trick we need to create a matrix literal with only one row; we wrote it as a one-column matrix and then transposed it. This is a valid alternative:

```
model::mvo(
    [1, 0.8, 0.6],
    aaa, aab, aad).setConstraints(
        matrix::rows([1, 0, -1]), [0], [int:1])
```

Linear Programming

The `model` class also provides a `simplex` method for solving linear programming problems. In a typical linear programming problem, we must maximise the value of a linear function like this:

$$40x_1 + 30x_2$$

All variables are implicitly considered non-negative, and some additional constraints must be satisfied:

$$x_1 + x_2 \leq 12$$
$$2x_1 + x_2 \leq 16$$

This problem can be solved using this code:

```
model::simplex([40, 30], [1, 1; 2, 1], [12, 16], [int: -1, -1])
```

The first parameter contains the coefficients from the objective function. The second parameter is a matrix with the left-hand side coefficients of the constraints, and the third parameter is the right-hand side of the constraint as a vector. Finally, the last

parameter contains the relational operators for each constraint. Since all constraints have the same sign, we can simplify the code like this:

```
model::simplex([40, 30], [1, 1; 2, 1], [12, 16], -1)
```

In both cases, the answer is a `SimplexModel` object that contains the optimal value and the coefficients for the optimal solution:

```
> model::simplex([40, 30], [1, 1; 2, 1], [12, 16], [int: -1, -1])
LP Model (2 variables)
Value: 400
Weights:
  4    8
```

This method always assumes that we want to maximise the value of the objective function. If you want to minimise the objective function, you could invert the sign of the coefficients:

```
model::simplex(-[12, 16], [1, 2; 1, 1], [40, 30], +1)
```

Note, however, that doing by this, you will get a negated value for the solution.

```
LP Model (2 variables)
Value: -400
Weights:
 20   10
```

You can fix this problem by using `simplexMin` instead and using the original coefficients in the objective function:

```
model::simplexMin([12, 16], [1, 2; 1, 1], [40, 30], +1)
```

Index

A

accumulators, 44
all, 31
any, 31
arrays, 5
autocorrelation, 38, 39, 40, 46, 55
autoregressive model, 47

B

beta, 16, 26
bool, 20

C

Cholesky, 84
comments, 8
comparisons, 14
complex
  conjugation, 15
  imaginary unit, 7
  vector, 57
conditionals, 20

D

date, 19
  operators, 20
  vector, 61
  vector literals, 61
DC, 64
definitions, 23
  functions, 25

E

elif, 20
erf, 16, 26
evd, 81
ewma, 40

F

Fast Fourier Transform, 64
  inverse, 64
  Wiener-Khinchin theorem, 46
fft, 64
Fibonacci, 63
functions
  definition, 25
  description, 26
  local, 30

G

gamma, 16
goodness of fit, 43, 49

I

if, 20
integer
  sequences, 71
  vector, 59
  vector literals, 60

L

lambdas, 31
  nested, 34
  parameters, 26
let, 29
  script-scoped, 29
linear model, 43
linearFit, 41
list comprehensions, 85
  quantifiers, 86

M

map, 31
matrix, 77
  cholesky, 84
  concatenation, 77
  evd, 81
  literals, 77
  LU factorisation, 83
  optimisations, 80
  transpose, 80
Mean Variance Optimiser, 3, 93
  additional constraints, 96
membership, 14
models
  AR, 40, 56, 67
  linearModel, 40, 56

MA, 40, 56, 67
simplex, 97
simplexMin, 98
moving average model, 49

N

ncdf, 33
Newton-Raphson, 18
nrandom, 17

O

operators, 13
complex conjugation, 15
date, 20
matrix transpose, 80

P

partial autocorrelation, 47
polynomials, 18
derivative, 19
splines, 90, 91
probit, 17

Q

quantifiers, 86

R

r2, 43, 49
random, 18
ranges, 14

S

sequences, 67
unfold, 73
until, 73
while, 69
series
ewma, 40
fit, 41
frequency, 37
set, 10
simplex, 97
splines, 89
area, 90
derivative, 90, 91
interacting with, 91

T

type
names, 26

U

undef, 23
unfold, 68, 73
until, 73

V

vectors, 53
autocorrelation, 55
complex, 57
concatenation, 53
date, 61
integer, 59
literals, 53, 60
optimisations, 54, 80
safe indexers, 63
slices, 63

W

while, 69